Software Requirements Specification

for

<URECAMatch>

Version 1.0

Prepared by <IAN TAN, ROBIN TEOH, NICHOLAS NG,

EDWIN TAN QI LUN, BENJAMIN NG >

SCEF GRP 4

<18/10/2026>

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: Class / Boundary Diagram 6
Appendix D: System Architecture Diagram

Revision History

Name	Date	Reason For Changes	Version
Robin	2/9/2026	LAB 1 Feedback (Data Dictionary, FR/NFR Identifiers)	0.1
	15/9/2026	LAB 2 (Data Dictionary, FR/NFR, Class Diagram, Dialog Map)	0.2
	17/10/2026	LAB 3	

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the requirements and specifications for URECAMatch (“Version 1.0”), a web-based portal designed to host the application and allocation process of the Undergraduate Research Experience on Campus (URECA) programme at Nanyang Technological University (NTU).

1.2 Document Conventions

<Describe any standards or typographical conventions that were followed when writing this SRS, such as fonts or highlighting that have special significance. For example, state whether priorities for higher-level requirements are assumed to be inherited by detailed requirements, or whether every requirement statement is to have its own priority.>

This SRS document adheres to a few standard formatting conventions for presentation consistency:

- Font Style
  - Header Text: Arial
  - Body Text: Times/Times New Roman
  -
- Requirement Identification
  - Each Functional Requirement (FR) or Use Case (UC) is uniquely indexed with its own code (eg. REQ-ROUND-1) for ease of reference.

- Versioning
  - Each revision of this document is logged in its Revision History section, along with reasons for each revision.

1.3 Intended Audience and Reading Suggestions

<Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this SRS contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.>

1.4 Product Scope

<Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies. If a separate vision and scope document is available, refer to it rather than duplicating its contents here.>

1.5 References

<List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.>

2. Overall Description

2.1 Product Perspective

URECAMatch is a new, self-contained web-based product. It is not a follow-on member of a product family, nor is it a component of a larger software system. It replaces the existing manual process for allocating students to URECA research projects, in which projects are published as a list of titles on the NTU website and students approach supervisors individually by e-mail. That process provides no project detail, no search or filtering capability, no visibility of remaining slots, and no means for either party to express ranked preferences. Section 1.4 describes the existing process and the objectives of its replacement in full.

The system operates independently of NTU institutional systems and neither reads from nor writes to them (Figure 2.1). It depends on four external services. Registration is restricted to NTU e-mail domains, and verification codes are sent to the user's NTU address. An e-mail delivery service sends verification codes and notifications. Cloud services host the application and provide its database, user authentication and file storage. A scheduler runs a daily update that advances the status of each matching round and times out offers that have not been answered. Supporting documents such as academic transcripts are uploaded by students and held in the system's own document storage rather than retrieved from institutional records. Integration with NTU single sign-on is desirable for a production deployment but is outside the scope of this release. Each interface is specified in Section 3.

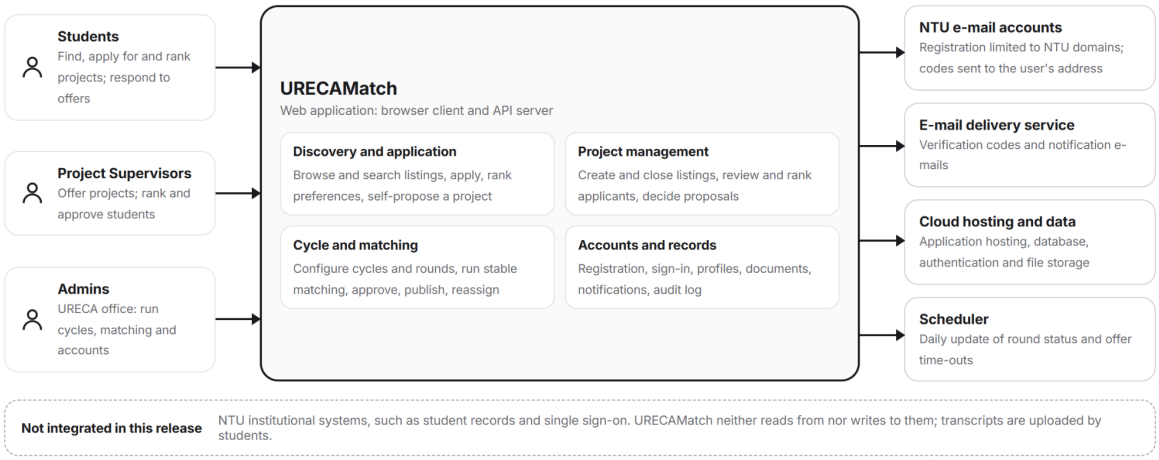


Figure 2.1 System overview

Allocation is organised into application cycles, normally one per academic year (Figure 2.2). Each cycle opens with a pre-application period, in which students may browse listings but not apply, followed by up to three matching rounds. In each round, students apply to projects and rank their applications during the application window; supervisors review and rank their applicants during the review window; an Admin then runs the matching process, supervisors approve or reject the proposed matches, and the Admin publishes the results. Students respond to their offers before the next round's review window begins. Students who remain unmatched and eligible may take part in the next round, so that the slots released by declined or expired offers can still be filled within the same cycle.

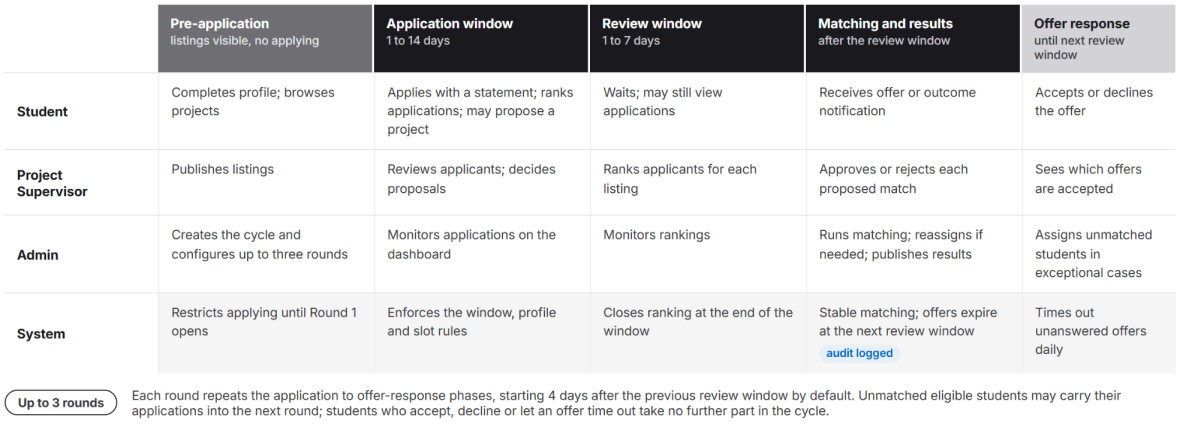


Figure 2.2 Application cycle and responsibilities

2.2 Product Functions

This section summarises the major functions that URECAMatch performs and the functions it allows each class of user to perform. Functions are grouped by the user role that initiates them. Detailed descriptions of each function, including its priority, stimulus and response sequences, and associated functional requirements, are provided in Section 4.

2.2.1 Use Case Diagram

The use case diagram in Figure 2.3 shows the actors that interact with URECAMatch and the use cases available to each. Student, Project Supervisor and Admin are specialisations of the generalised User actor, which owns the use cases common to all roles. Two system actors also take part: a scheduler, which times out match offers that

are not answered before they expire, and an e-mail delivery service, which receives verification codes and notification e-mails for delivery. Each use case corresponds to the system feature of the same name in Section 4, and its full description is provided in the Use Case Descriptions document (Reference 1).

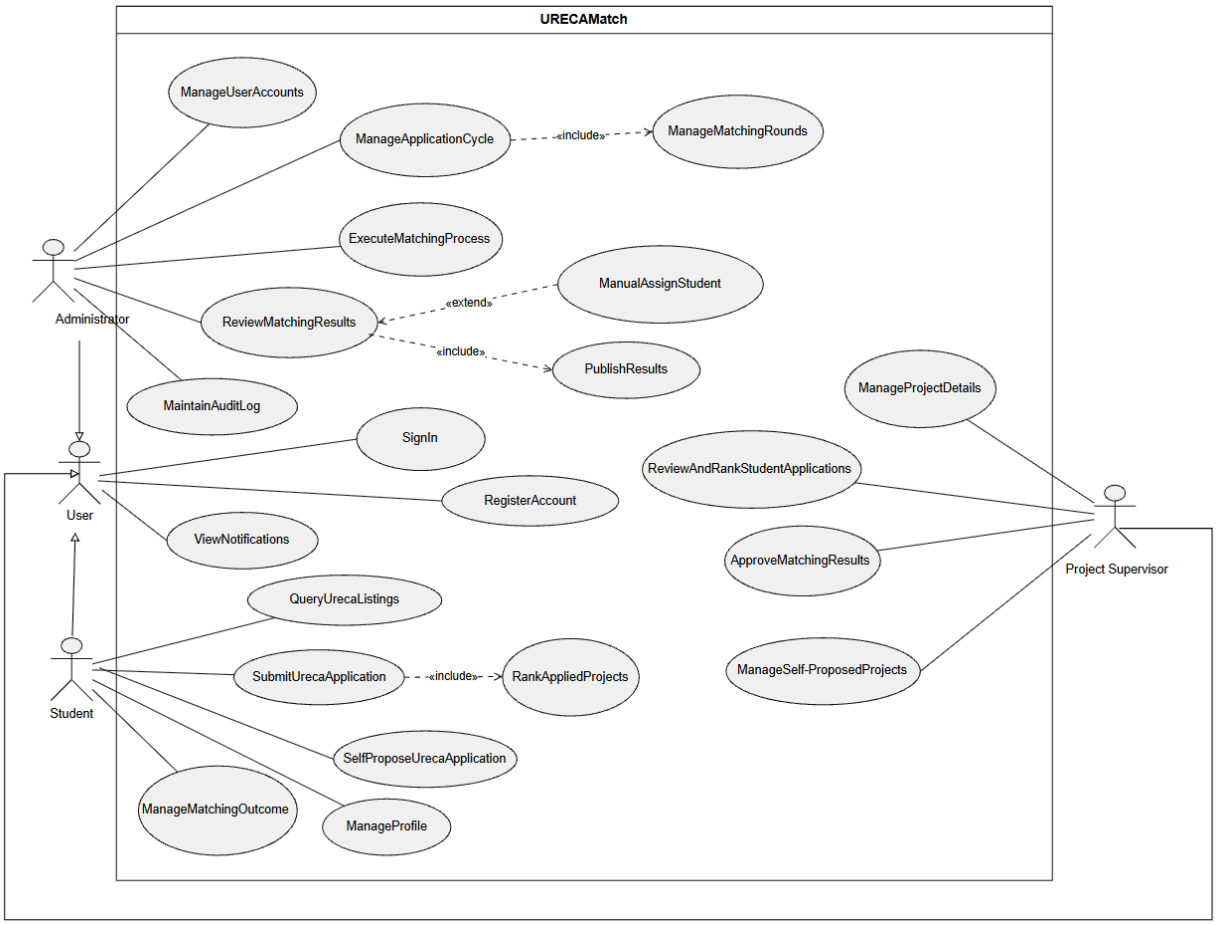


Figure 2.3 Use case diagram

2.2.2 Major Product Functions

URECAMatch allows its users to perform the following functions:

- **All users**
  - Register an account as a Student or Project Supervisor and verify it with a code sent to an NTU e-mail address; Admin accounts are created by an Admin
  - Sign in, sign out, and view notifications

- **Students**
  - Complete and maintain a profile, including completed modules, research interests and supporting documents
  - Browse, search and filter project listings by keyword, school, topic and supervisor
  - Apply to listings with a statement of interest and supporting documents, and withdraw an application while it is under review
  - Rank applications in order of preference
  - Propose a project directly to a chosen supervisor
  - View published results and accept or decline a match offer
- **Project Supervisors**
  - Create, publish, update and close project listings, or save them as drafts
  - Review applicants and their supporting documents, and rank applicants by preference
  - Approve or reject the matches proposed for their projects
  - Review student proposals and accept or decline them
- **Admins**
  - Create, change, close and reopen application cycles
  - Configure up to three sequential matching rounds and whether unmatched applications carry over
  - Execute the matching process, review the results and publish them
  - Reassign a match, assign an unmatched student manually, or unassign a student, each with a recorded justification
  - Create, update and deactivate user accounts
  - View the audit log of all administrative operations
- **System**
  - Notify users in the application and by e-mail when an event concerns them
  - Advance round status and time out unanswered offers each day

2.2.3 Class Diagram

The class diagram in Figure 2.4 shows the principal entity classes maintained by URECAMatch, their attributes, and the relationships between them. User is specialised into Student, ProjectSupervisor and Admin, and Application into ListingApplication and SelfProposedApplication. An ApplicationCycle comprises one to three MatchingRounds; each round collects listing applications and applicant rankings and produces match results, each of which allocates one student to one project listing. Notifications and verification codes are associated with the user they concern, and audit log entries with the Admin who initiated them.

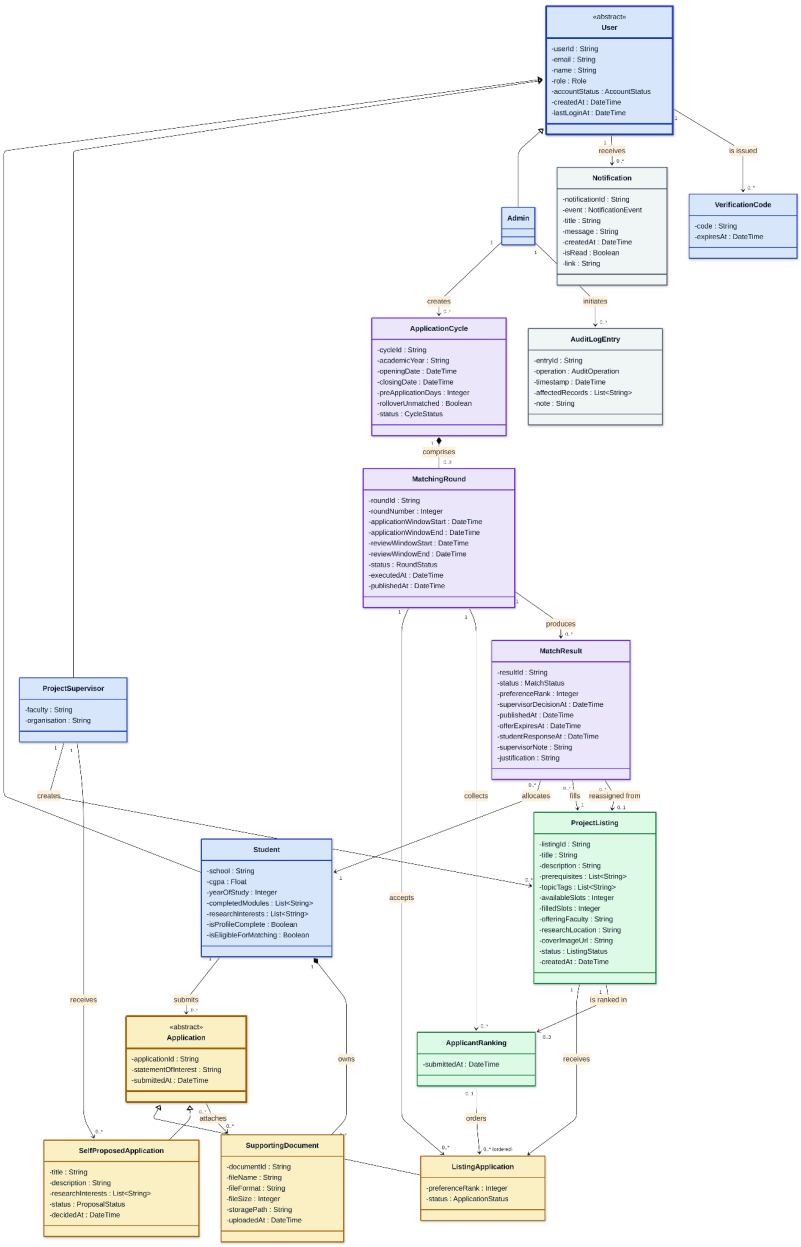


Figure 2.4 Class diagram

2.3 User Classes and Characteristics

Users of the platform fall into three classes, which differ in number, frequency of use, privilege and the functions they need (Figure 2.5).

Students are undergraduates of NTU seeking a URECA research placement, typically in their second or third year of study. They are the largest user class. All are familiar with web applications, but their technical expertise varies and none can be assumed to have prior

experience of a preference-based allocation system. Usage is concentrated and infrequent: a student interacts with the system heavily during the application window of each matching round and when results are published, and rarely at other times. Students hold the lowest privilege level, with access restricted to their own profile, applications and match results. They use the discovery, application, ranking and match response functions, and no administrative function.

Project Supervisors are academic staff who offer research projects and select students for them. They are experienced professionals with high educational attainment but no assumed familiarity with the system, and their available time is limited. Usage is periodic rather than continuous, consisting of listing projects before a cycle opens and of applicant review, ranking and match approval within each round. Applicant review must therefore be achievable in a single sitting without recourse to external tools, which is the basis of the requirement that documents be viewable within the system. Supervisors may access only the projects they own and the applicants to those projects.

Administrators are staff of the URECA office responsible for overseeing the allocation process. They are the smallest user class, are trained in the operation of the system, and use it throughout a cycle rather than at particular points. They hold the highest privilege level and are the only class able to configure application cycles and matching rounds, execute the matching process, publish results, override an allocation manually and manage user accounts. The audit and override functions pertain exclusively to this class.

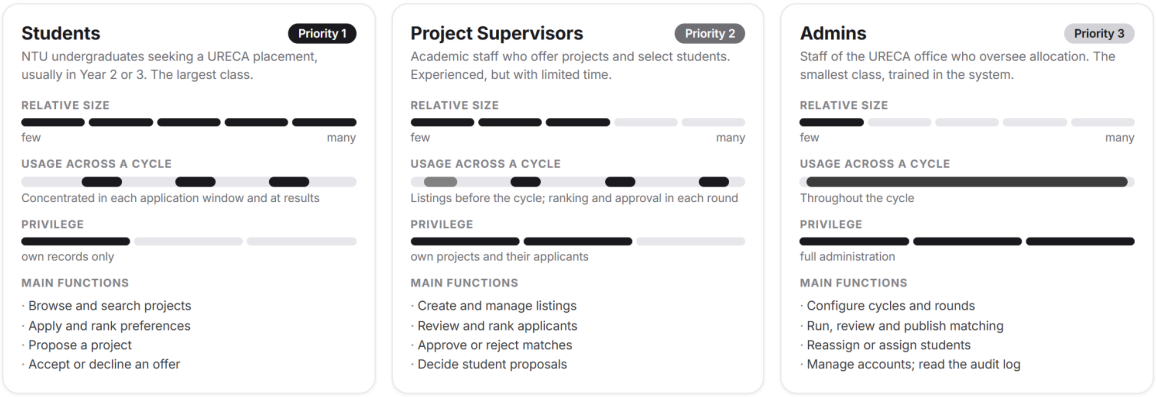


Figure 2.5 User classes and characteristics

Students are the most important user class to satisfy. They are the most numerous, they are the group whose experience of the existing process the system exists to improve, and they have the least tolerance for a system that is difficult to use, since they have no obligation to persist with one. Project Supervisors are the second most important. Their participation is essential, as a matching round cannot produce a valid allocation without supervisor rankings, but they are a smaller group with a direct interest in the outcome and are correspondingly more likely to complete a task that is not effortless. Administrators are the least important class to satisfy in terms of interface quality. They are few in number, are trained in the system and use it as part of their duties, so their functions may prioritise correctness, completeness and auditability over ease of use.

2.4 Operating Environment

URECAMatch is a web application, and no software is installed on the user's computer. Users access it through a current version of Chrome, Edge, Safari or Firefox on a desktop or laptop computer running any operating system those browsers support, in a browser window at least 1200 px wide. Tablets and smartphones are not supported in this release.

The web client, built with Next.js 16, and the API server, built with Python 3.12 and FastAPI, are hosted on Vercel in its Singapore region (sin1). The database, user authentication and file storage are provided by Supabase, a managed PostgreSQL service, and e-mail is delivered through Resend. The system must coexist with the browser extensions users may have installed and with NTU's e-mail service, through which verification codes and notifications are received. The supported environment and every external interface are specified in Section 3.

2.5 Design and Implementation Constraints

The following constraints limit the options available to the developers:

- **Technology** The web client is implemented in Next.js with TypeScript, the API server in Python with FastAPI, persistence, authentication and file storage on Supabase, and hosting on Vercel. These were chosen to match the team's existing expertise.
- **Architecture** The system follows a layered Boundary-Control-Entity architecture. Every business rule is enforced in the API server, not only in the browser; all data is accessed through a single Store interface; and the generated API contract is the single definition shared by the web client and the server.
- **Hosting** The API runs as serverless functions, so persistent state must be held in the database rather than in server memory, and each request must complete within the hosting provider's time limit.
- **Service limits** The free tiers of Supabase, Vercel and Resend limit database size, function execution time and the number of e-mails that can be sent each day.
- **Data protection** Personal data is handled in accordance with Singapore's Personal Data Protection Act (Section 5.3.3). Student documents are stored privately and released only to authorised users through short-lived links.
- **Security** Registration is restricted to NTU e-mail domains, passwords are handled only by the authentication service, and service keys are never sent to the browser.
- **Language** The interface is in English, and dates and times are shown in Singapore time.

- **Process** The product is delivered within one semester by a team of five. All changes are made through reviewed pull requests, and every change must pass automated linting, tests, type checks and a production build before it is merged. Code and tests are tagged with the requirement identifiers they implement.

2.6 User Documentation

No separate user manual is delivered. Users are guided within the interface itself: every screen carries a short description under its title, form fields carry hints, empty lists explain what will appear and how to proceed, and every error message states how to correct the problem (Section 3.1.1).

For developers and maintainers, the following documentation is delivered with the source code and maintained alongside it in Markdown:

- **Setup guide** How to install, configure and run the web client and the API server, in mock and live modes (docs/SETUP.md).
- **Contributing guide** Branching, review and testing practices for the team (docs/CONTRIBUTING.md).
- **Architecture** The layers, components and design patterns of the system (docs/architecture.md).
- **API specification** The OpenAPI document generated from the code (shared/openapi.json), browsable at /docs on a running API server.
- **Traceability matrix** The mapping from each requirement to its endpoint, screen and test (docs/traceability.md).

2.7 Assumptions and Dependencies

The requirements in this document rest on the following assumptions. If any of them proves incorrect, the affected requirements must be reviewed.

- **A-1** Every student and supervisor has an active NTU e-mail account that can receive messages from the system.
- **A-2** Supervisors rank their applicants before each review window closes. Applicants who are not ranked cannot be matched to that supervisor's project.
- **A-3** Documents uploaded by students are genuine. The system does not verify transcripts against NTU records.
- **A-4** At most one application cycle is active at a time, and each cycle is of a size similar to the current URECA programme: about 1,300 project listings from about 480 supervisors, based on the currently published URECA project list.

- **A-5** Users access the system from a supported desktop browser with a reliable internet connection.

The project also depends on the following external factors:

- **D-1** Supabase remains available for the database, user authentication and file storage.
- **D-2** Vercel remains available for hosting the application and running the daily scheduled update.
- **D-3** Resend delivers e-mail promptly, and NTU's e-mail service does not filter it as spam. The notification latency in NFR 5.1.2 depends on both.
- **D-4** The open-source frameworks on which the system is built, including Next.js, React, FastAPI and Pydantic, remain supported. A major upgrade of any of them may require changes to the code.

3. External Interface Requirements

This section specifies the interfaces between URECAMatch and the world outside it: its users (3.1), the hardware it runs on (3.2), the external software it depends on (3.3) and the communication between them (3.4).

3.1 User Interfaces

URECAMatch is used through a web browser by three groups of users: Students, Project Supervisors and Admins. This subsection specifies the logical characteristics of the interface between the system and these users: the conventions that apply to every screen, the visual style, the shared components, the way results and errors are reported, accessibility, and the design of each screen. The visual style and components are defined once in a shared stylesheet and component library, so that every screen looks and behaves the same; the flow between screens is specified by the dialog map in Appendix B.

3.1.1 General Conventions

Each user signs in once and is then shown only the functions of their role. A sidebar on every signed-in screen carries the navigation for that role, the user's name and role, and the sign-out control (Figure 3.1). After sign-in, Students arrive at Browse projects, Project Supervisors at My projects and Admins at the Dashboard. Counts of unread notifications and of items awaiting the user, such as proposals or match approvals, appear as badges beside the relevant menu item, so that pending work is visible from any screen.

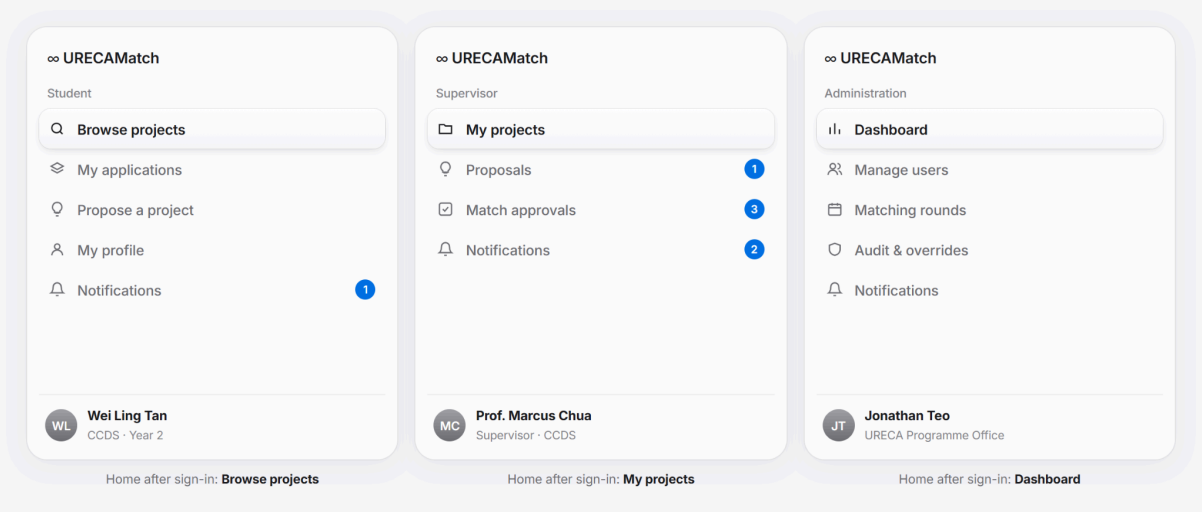


Figure 3.1 Navigation by role

Every screen follows the same structure (Figure 3.2): a page header with the screen title and a one-line description, a back link on screens reached from another screen, an optional deadline for the current round, and the content of the screen. Where a list can be long, search and filters sit directly above it. The interface is in English. Dates are shown as day and month, for example 10 Sep, and times are shown in Singapore time and labelled SGT, so that every deadline is unambiguous.

The system has no separate help pages. Guidance is given where it is needed instead: each screen has a description under its title, form fields carry short hints, empty lists explain what will appear and how to proceed, and every error message states how to correct the problem (3.1.6).

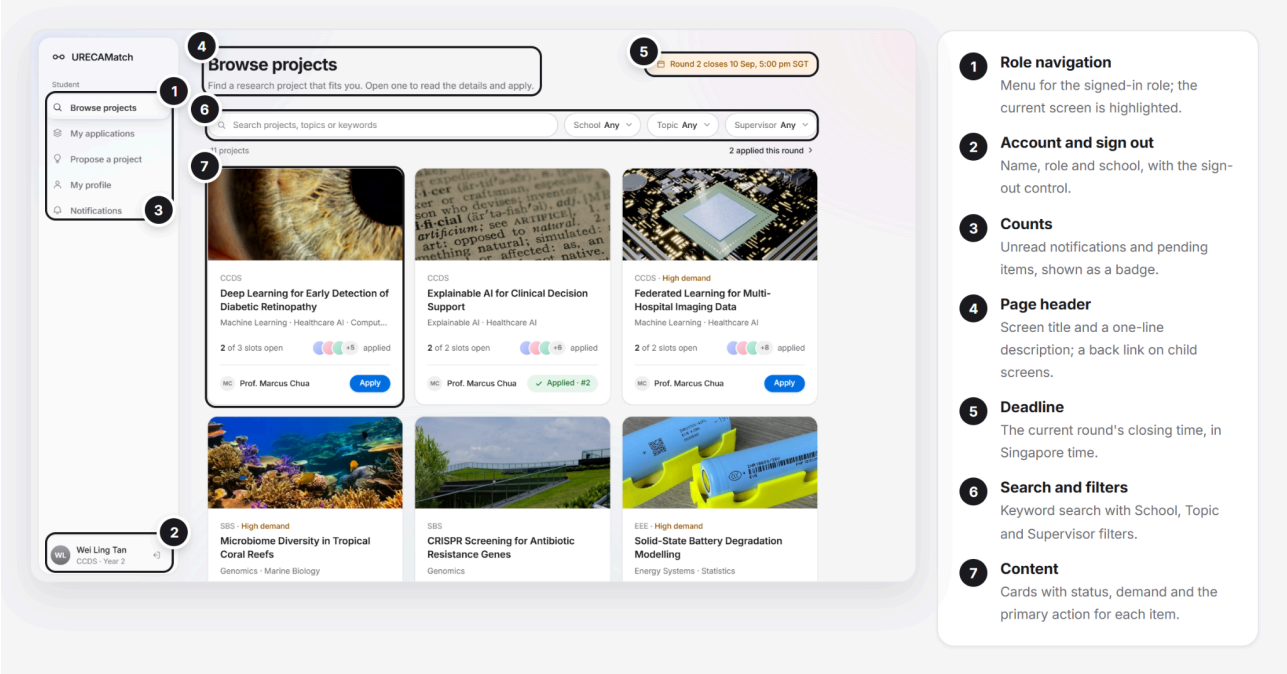


Figure 3.2 Anatomy of a screen

3.1.2 Colour

The palette is deliberately neutral, so that colour carries meaning rather than decoration (Figure 3.3). Greys and white draw the structure of every screen. A single accent blue marks what the user can act on: buttons, links, the keyboard focus ring and count badges. Three semantic colours express state: green for success and approval, amber for caution, waiting and deadlines, and red for errors, refusals and destructive actions. Each semantic colour has a darker ink shade, used for text, and a light soft shade, used for backgrounds, and text placed on a soft shade always uses the ink shade of the same colour. The interface uses a light colour scheme only.

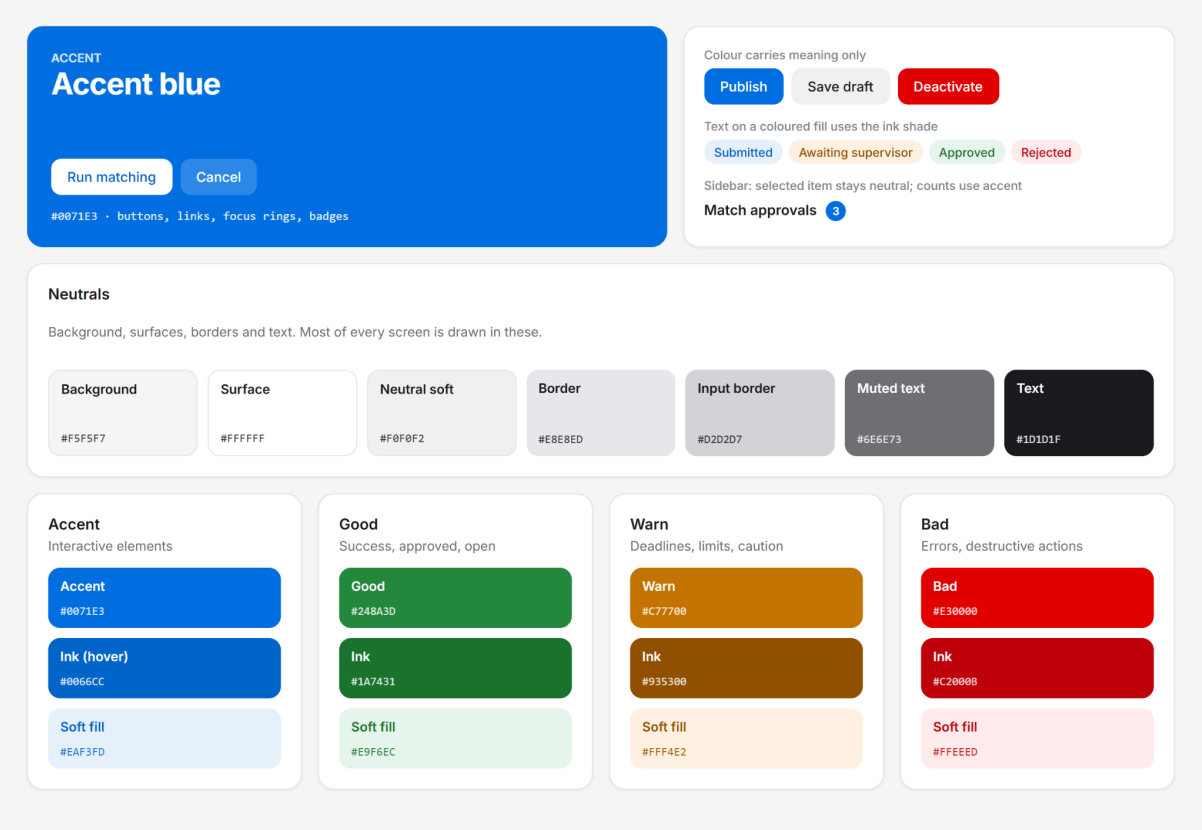


Figure 3.3 Colour palette

3.1.3 Typography

All text is set in one typeface, Inter, and hierarchy is expressed through size and weight rather than through different typefaces (Figure 3.4). Body text is 14 px with a line height of 1.5, page titles are 28 px bold, and no text is smaller than 11 px. Numbers that are read in columns or compared, such as dates, CGPA and slot counts, use tabular numerals so that their digits align. Identifiers such as module codes are shown in the system monospace font.

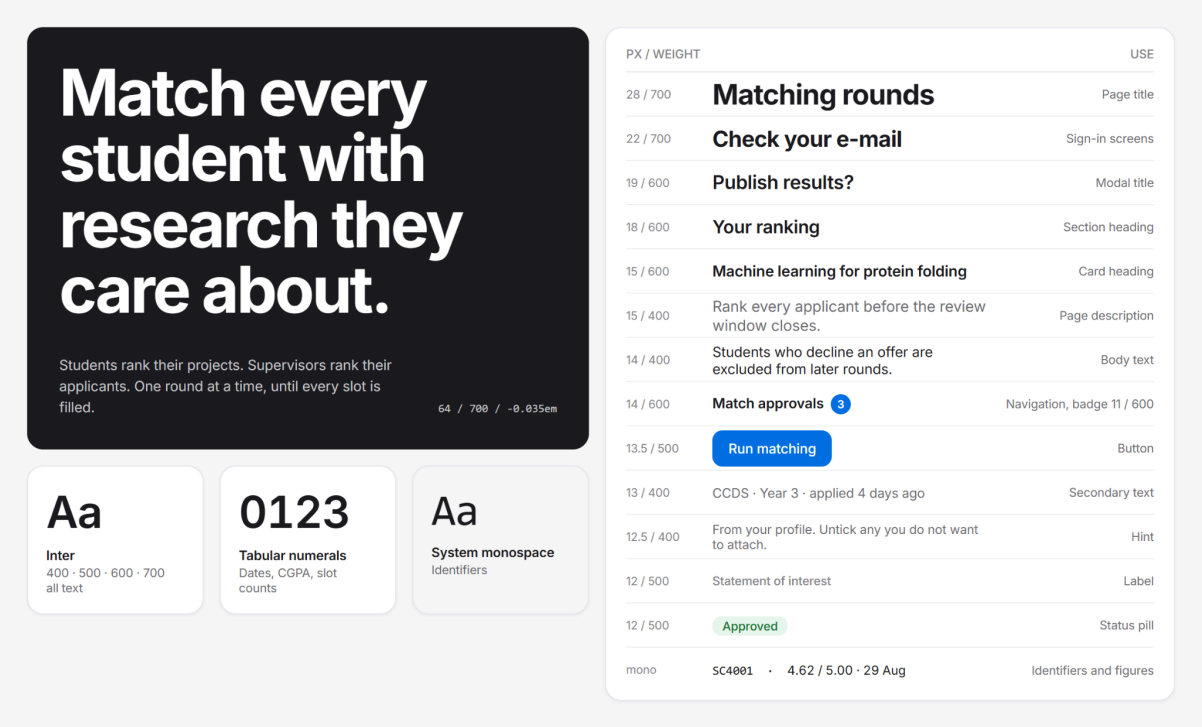


Figure 3.4 Typography

3.1.4 Layout, Shape and Interaction

All signed-in screens share one shell (Figure 3.5). A 240 px sidebar floats 12 px from the window edge, and the content region beside it is at most 1080 px wide. The interface is designed for a browser window at least 1200 px wide and aligns to a 4 px spacing grid. Corners are rounded according to the type of element, from 8 px for small icon controls to 20 px for modals and the sidebar, so that elements of the same kind always share the same shape.

Two overlays are used, each for one purpose. A drawer, 460 px wide, slides in from the right to show supporting detail without leaving the current screen, such as an applicant's profile and documents while a Supervisor ranks applicants. A modal, 520 px wide and centred, asks the user to confirm an action. Both dim the screen behind them and close when the user presses Escape or clicks outside them. Buttons come in three heights, 44, 36 and 28 px, and every control offers a target of at least 28 by 28 px.

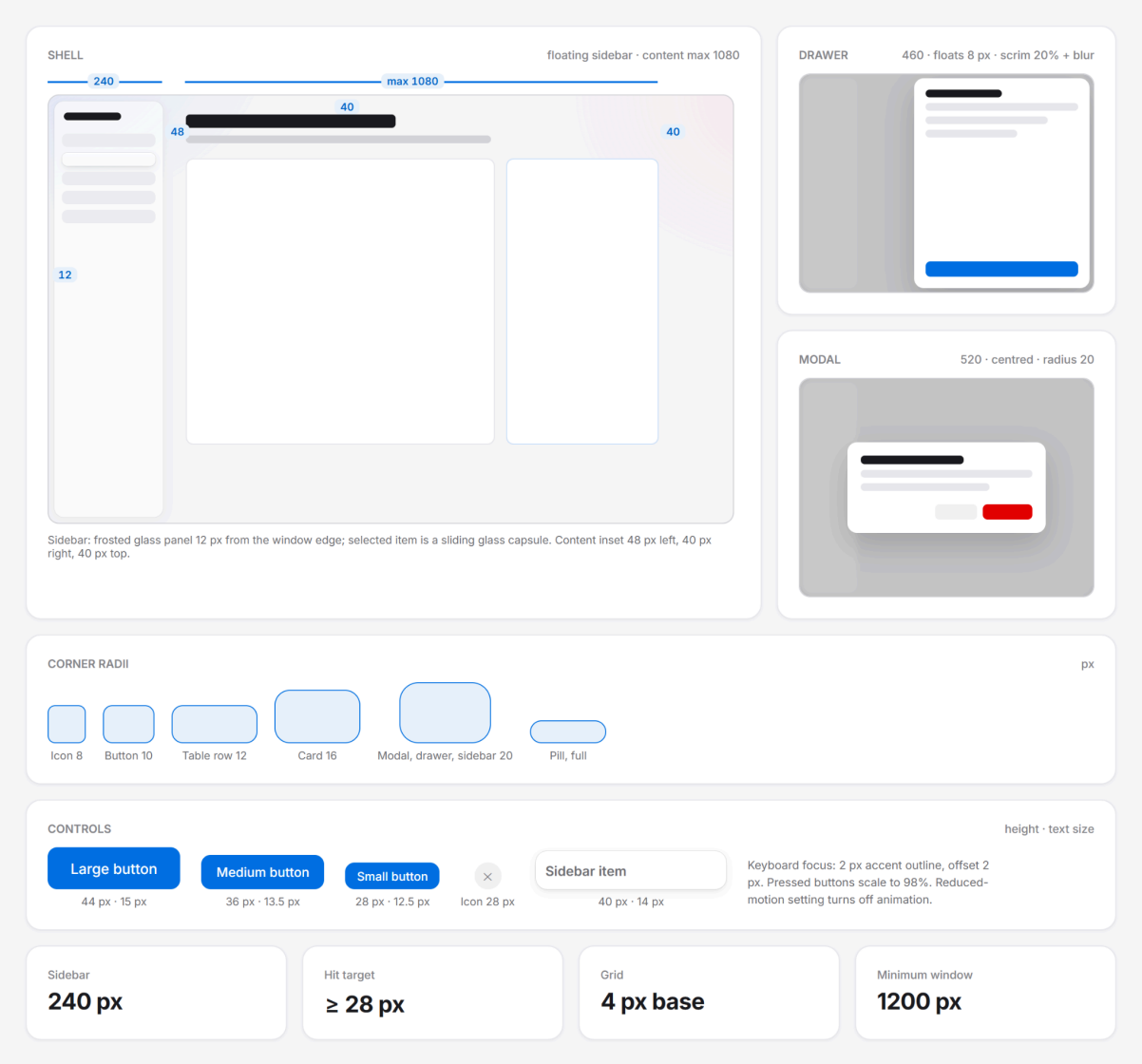


Figure 3.5 Layout, shape and controls

3.1.5 Common Components

Every screen is assembled from one shared library of components (Figure 3.6). A rule about appearance or behaviour, such as how a field error is shown or how a status is coloured, is therefore implemented once and applies to every screen that uses the component. The library provides buttons in five styles, text inputs and searchable pickers, tag inputs for topics and prerequisites, filter chips, status pills, deadline and rank badges, toggles and steppers, file upload, statistic cards and progress bars, the drawer and modal, and an in-browser PDF viewer. A new pattern is added to the library before it is used on a screen, so that the interface stays consistent as it grows.

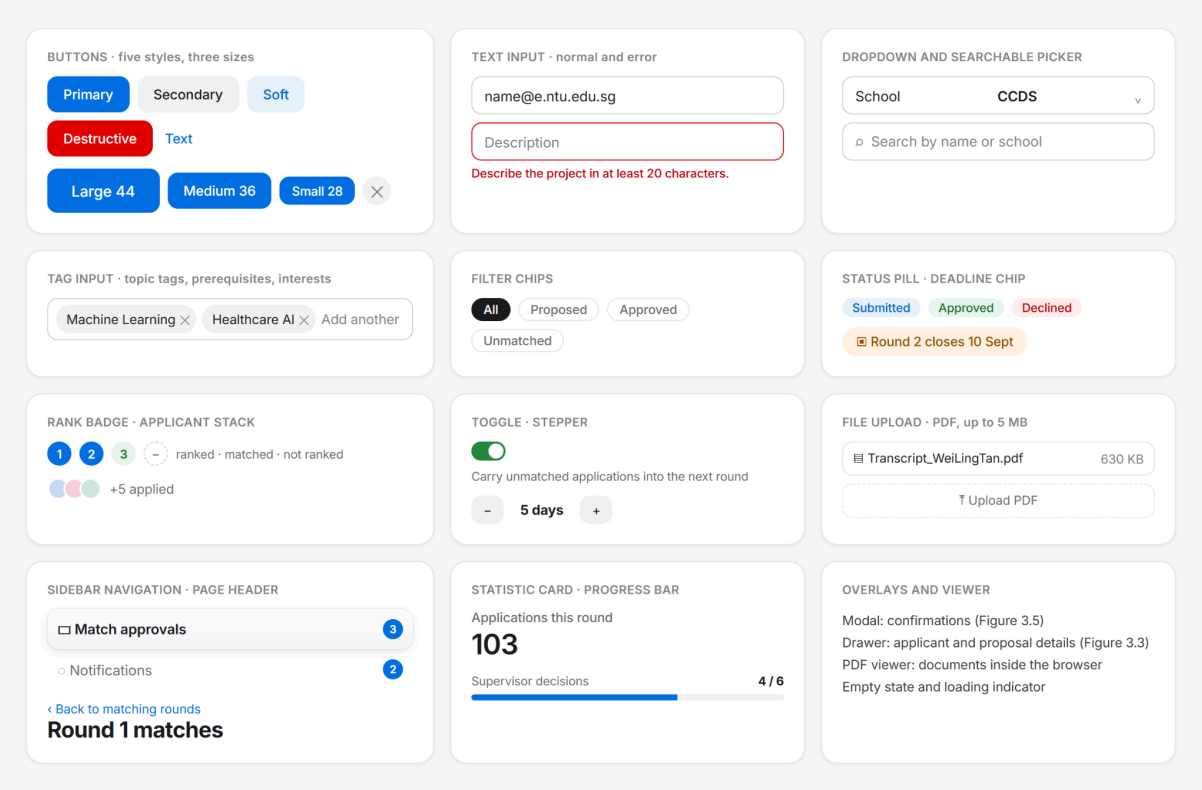


Figure 3.6 Common components

3.1.6 Messages and Status Labels

Every action a user takes produces a visible result, and every message is written in plain English, states what happened and, for a failure, says how to put it right (NFR 5.4.2). Messages take five forms (Figure 3.7). When the server rejects an input, the field at fault is outlined in red and the message appears directly below it. Other outcomes appear as a banner at the top of the content area: green when a change is saved, amber for a deadline or limit, red for a failure that is not tied to one field, and blue for guidance on the next step. Before any action that cannot be undone, such as closing a listing, withdrawing an application, accepting or declining an offer, deactivating a user or publishing results, a modal states the consequence and asks for confirmation. A list with no items explains why and offers the next step. When a session expires after 30 minutes without activity, the user is returned to the sign-in page with the reason.

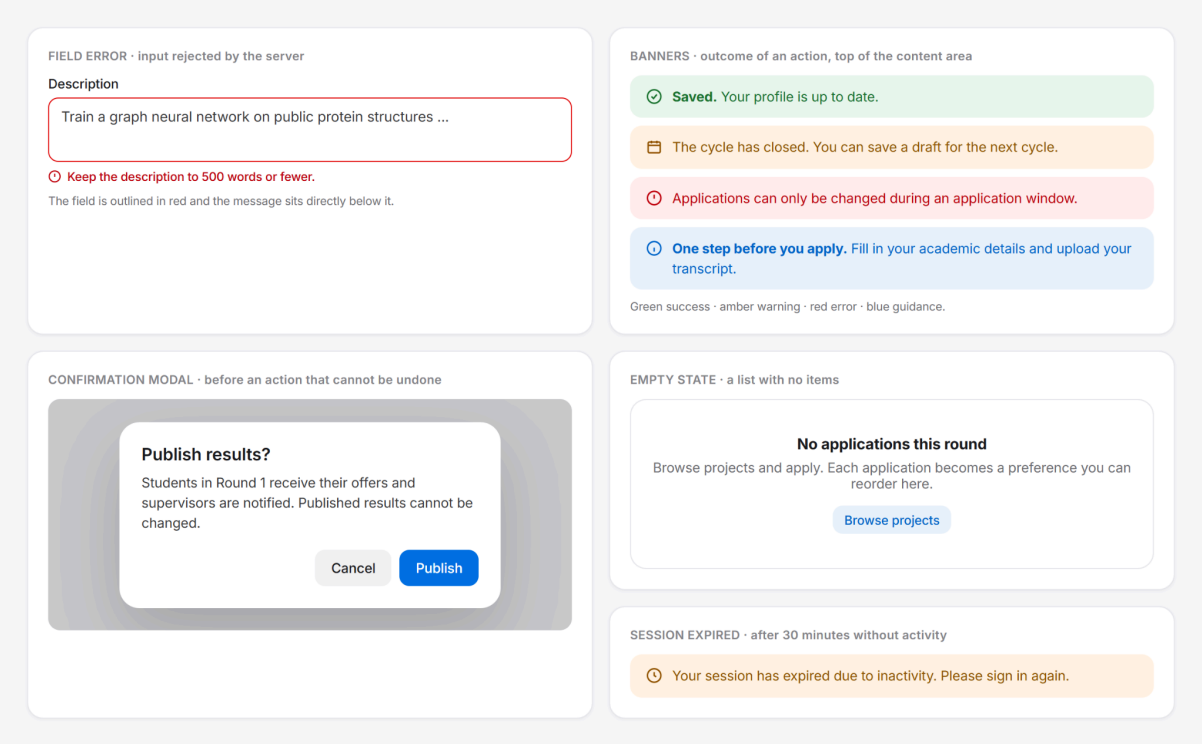


Figure 3.7 Messages and feedback

The status of every record, whether an application, match, proposal, listing, round or account, is shown with one fixed label and colour on every screen (Figure 3.8). The colour follows the meaning of the status: blue for in progress, green for success, amber for waiting or caution, red for ended or refused, and grey for inactive. A status therefore reads the same wherever it appears, and the label shown to users is always the same for the same stored value.

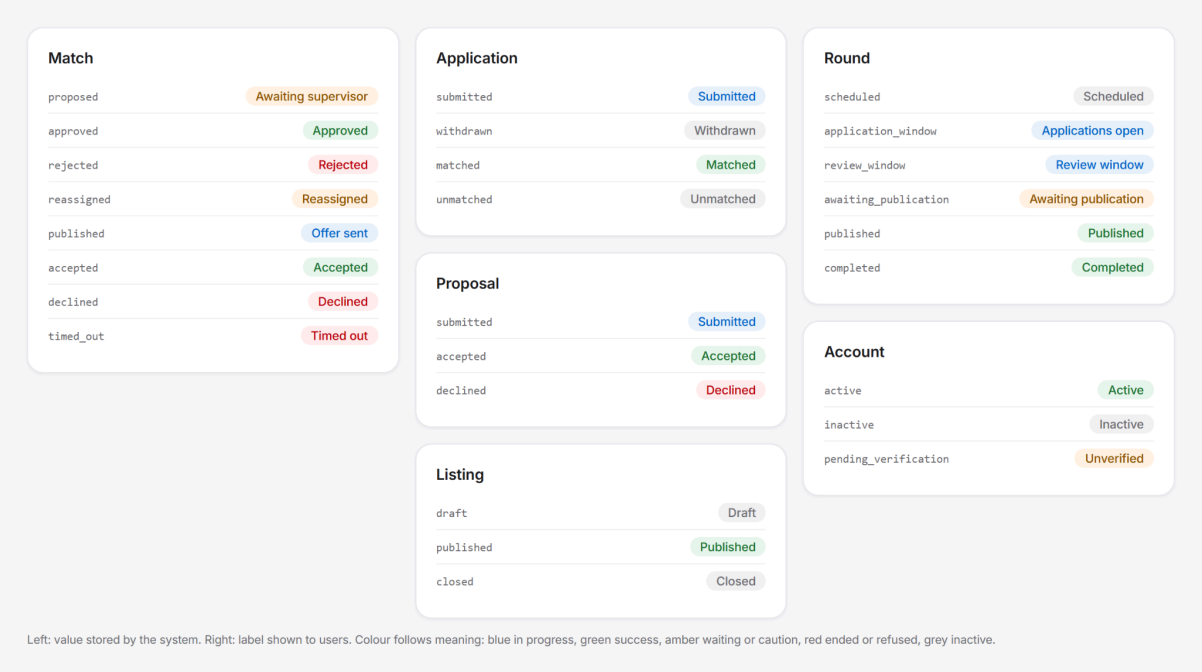


Figure 3.8 Status labels

3.1.7 Accessibility

The interface is designed to WCAG 2.2 Level AA. Text meets the contrast ratio of 4.5:1 against its background, with the exception of the faint grey used for hints and placeholders, which is to be darkened to meet it. Colour is never the only means of conveying information: every coloured status also carries a text label and every error also carries an icon. Every control can be reached and operated with the keyboard and shows a visible focus outline; forms submit with Enter, overlays close with Escape, and the dropdown menus follow the listbox pattern with arrow-key navigation. Icon-only controls carry a text label for screen readers, and the page language is declared as English. When the operating system requests reduced motion, animations and transitions are turned off.

3.1.8 Screen Designs

Figures 3.9 to 3.32 show every screen of the final prototype, grouped by user role and captured at a window size of 1440 by 900 px using the demonstration data. Each screen is named as in the dialog map (Appendix B), which specifies how the user moves between screens. Table 3.1 links each screen to its route and to the system feature in section 4 that it implements, so that every screen can be traced to its requirements.

Table 3.1 Screen index

Figure	Screen and route	User	Section 4 feature
3.9	SignInPage /sign-in	All	4.20 SignIn
3.10	CreateAccountPage /create-account	All	4.19 RegisterAccount

Figure	Screen and route	User	Section 4 feature
3.11	EmailVerificationPage /verify	All	4.19 RegisterAccount
3.12	CompleteProfilePage /profile?complete=1	Student	4.14 ManageProfile
3.13	BrowseProjectPage /browse	Student	4.9 QueryUrecaListings
3.14	ProjectDetailPage /projects/{id}	Student	4.9 QueryUrecaListings
3.15	ApplyProjectPage /apply/{id}	Student	4.10 SubmitUrecaApplication
3.16	ProposeProjectPage /propose	Student	4.12 SelfProposeUrecaApplication
3.17	ProposalSuccessPage /propose (after sending)	Student	4.12 SelfProposeUrecaApplication
3.18	MyApplicationPage /my-applications	Student	4.11 RankAppliedProjects 4.13 ManageMatchingOutcome
3.19	MyProfilePage /profile	Student	4.14 ManageProfile
3.20	NotificationPage /notifications	All	4.21 ViewNotifications
3.21	SupervisorProjectsPage /projects	Supervisor	4.15 ManageProjectDetails
3.22	CreateProjectPage /projects/new	Supervisor	4.15 ManageProjectDetails
3.23	EditProjectPage /projects/{id}/edit	Supervisor	4.15 ManageProjectDetails
3.24	ApplicantAndRankingsPage /projects/{id}/applicants	Supervisor	4.16 ReviewAndRankStudentApplications
3.25	StudentProposalsPage /proposals	Supervisor	4.18 ManageSelf-ProposedProjects
3.26	StudentProposalsPage, confirmation /proposals	Supervisor	4.18 ManageSelf-ProposedProjects
3.27	MatchApprovalsPage /approvals	Supervisor	4.17 ApproveMatchingResults
3.28	AdminDashboardPage /dashboard	Admin	4.1 to 4.8, overview
3.29	ManageUsersPage /users	Admin	4.8 ManageUserAccounts
3.30	MatchingRoundsPage /rounds	Admin	4.1 ManageApplicationCycle 4.2 ManageMatchingRounds 4.3 ExecuteMatchingProcess
3.31	MatchingAwaitPublishPage /rounds/{id}/matches	Admin	4.4 ReviewMatchingResults 4.5 PublishResults 4.6 ManualAssignStudent
3.32	AuditOverridesPage /audit	Admin	4.7 MaintainAuditLog

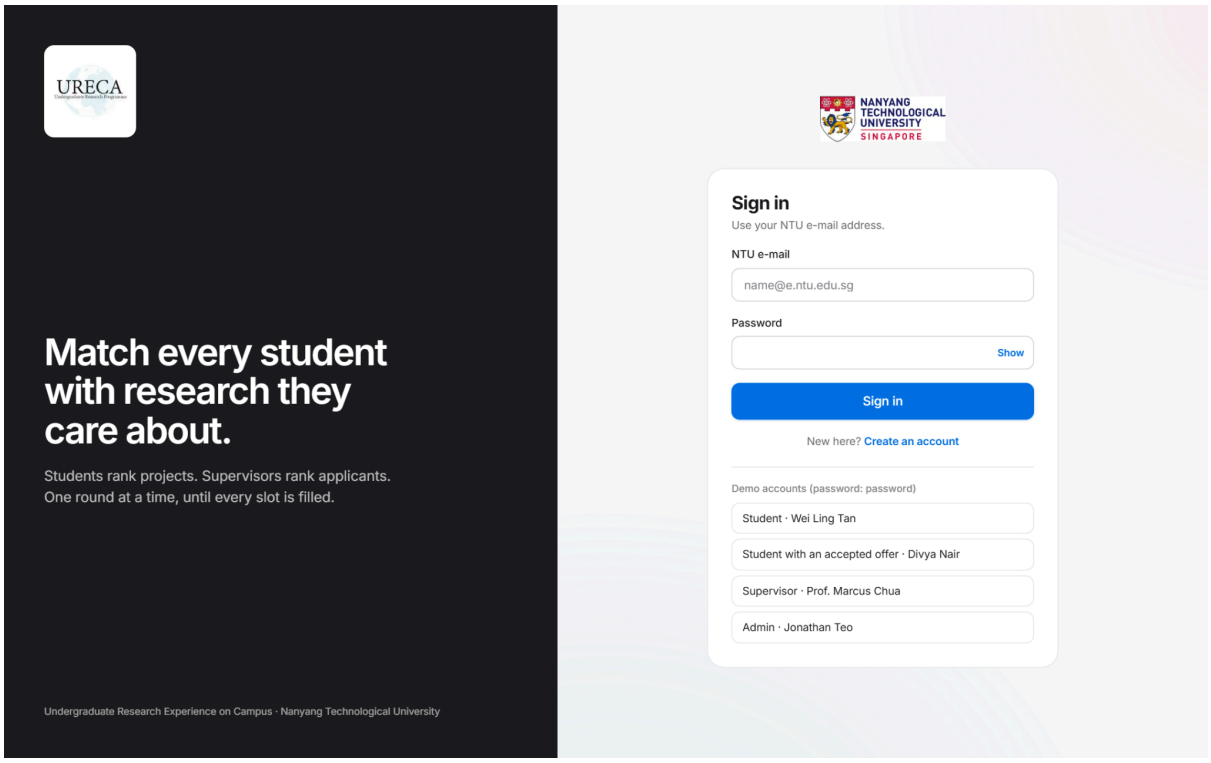


Figure 3.9 SignInPage

Sign-in with NTU e-mail and password. The password can be revealed. Links to account creation. The prototype build also lists the demo accounts.

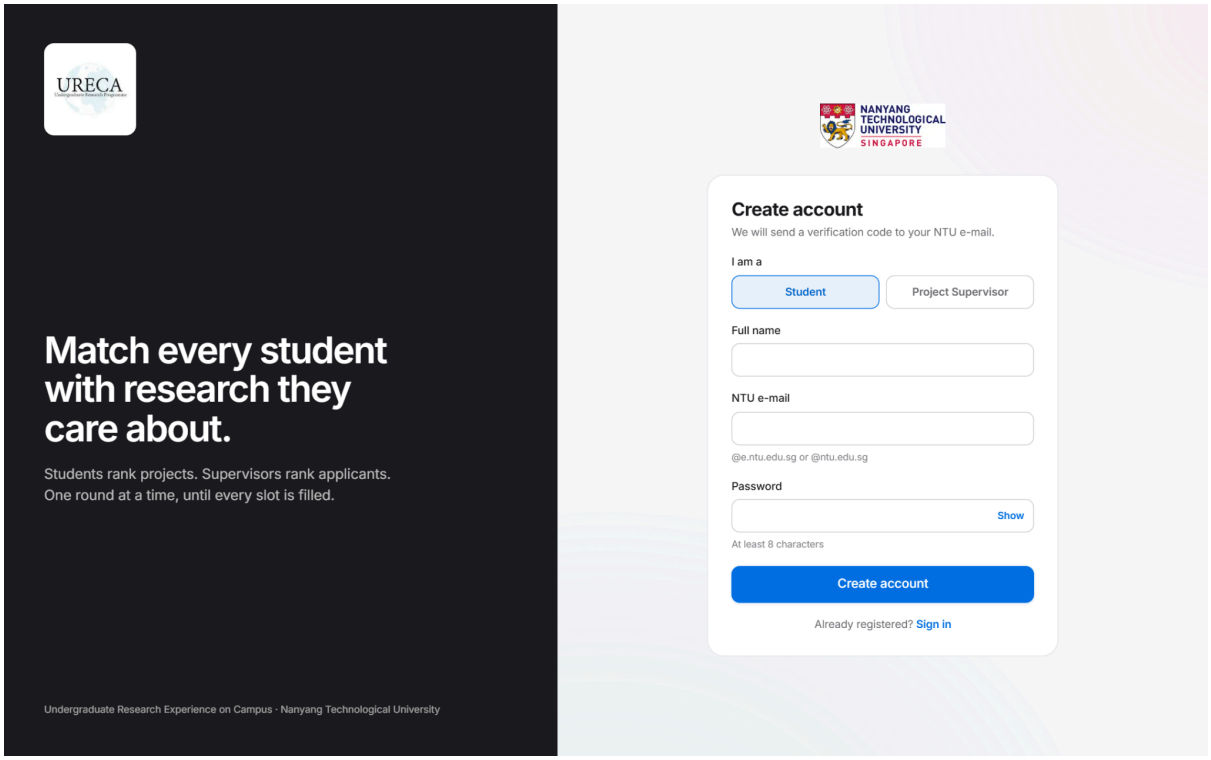


Figure 3.10 CreateAccountPage

Choice of Student or Project Supervisor, name, NTU e-mail and a password of at least 8 characters.

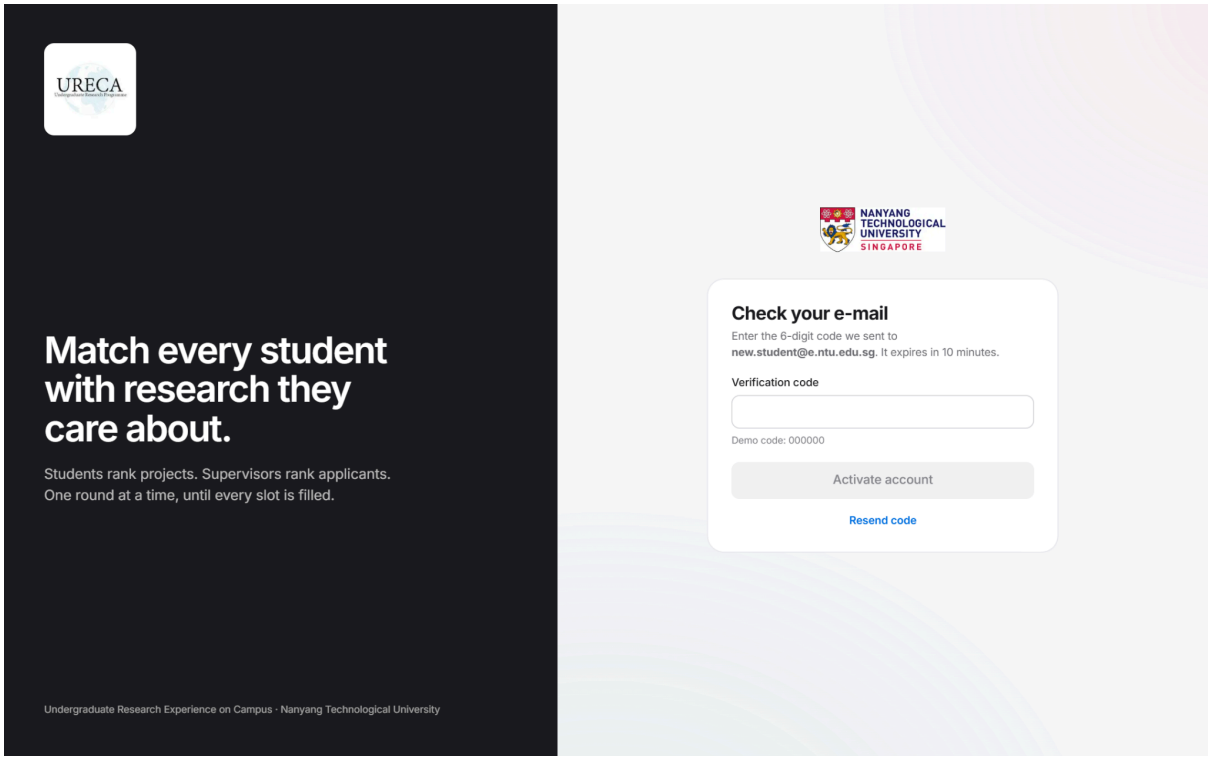


Figure 3.11 EmailVerificationPage

Entry of the 6-digit code sent by e-mail, which expires after 10 minutes. A new code can be requested.

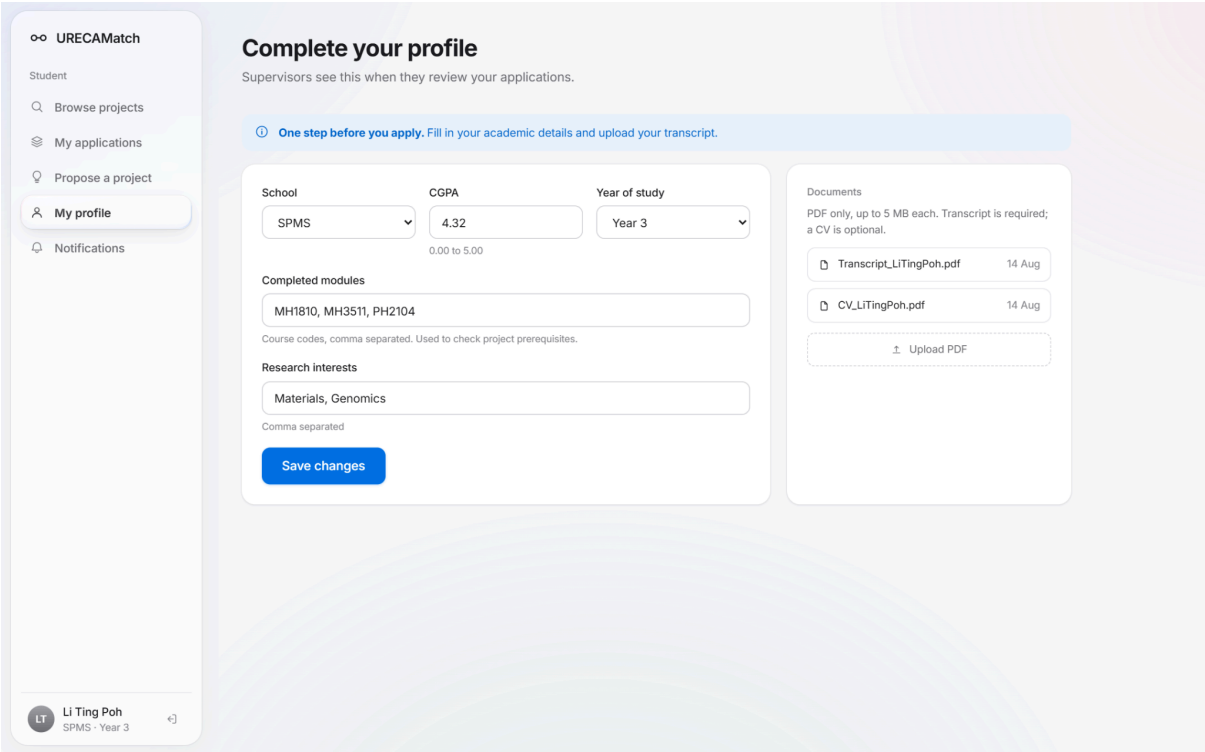


Figure 3.12 CompleteProfilePage

Shown after first sign-in, and in place of any other screen, until the profile is complete: school, CGPA, year, modules, interests and transcript.

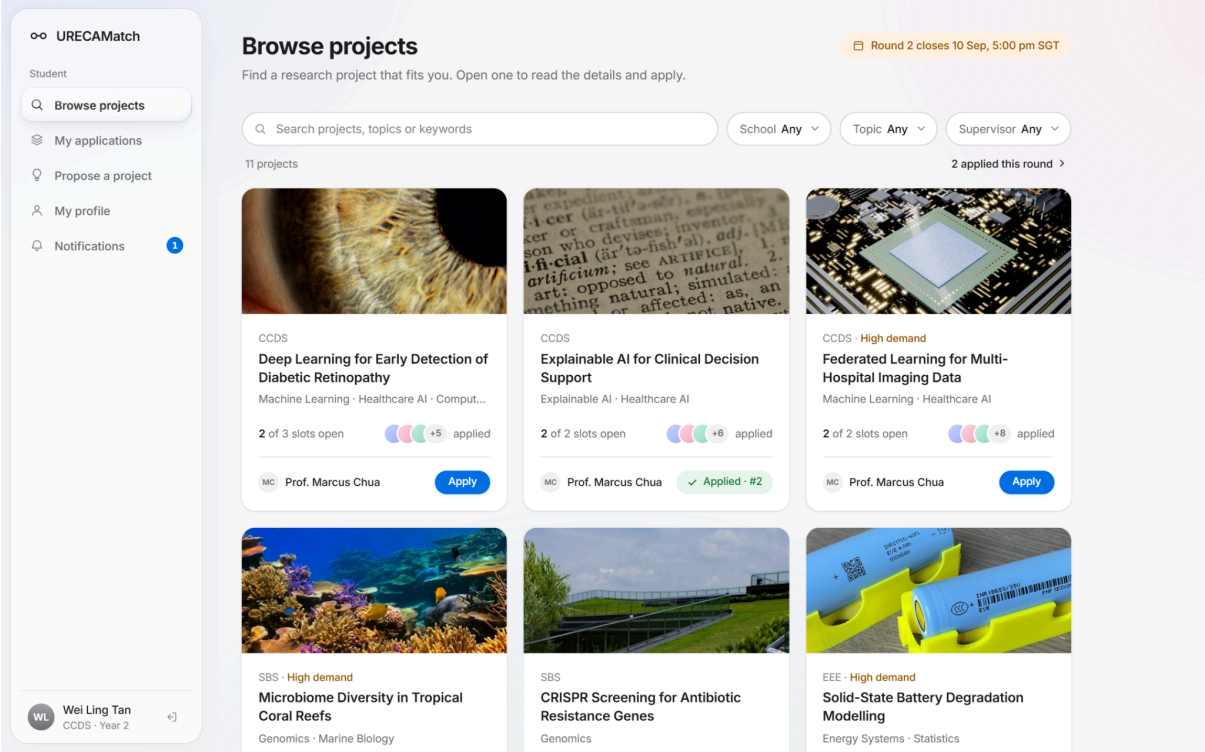


Figure 3.13 BrowseProjectPage

Search with School, Topic and Supervisor filters. Each card shows open slots, the number of applicants and a High demand label. The round deadline is shown top right.

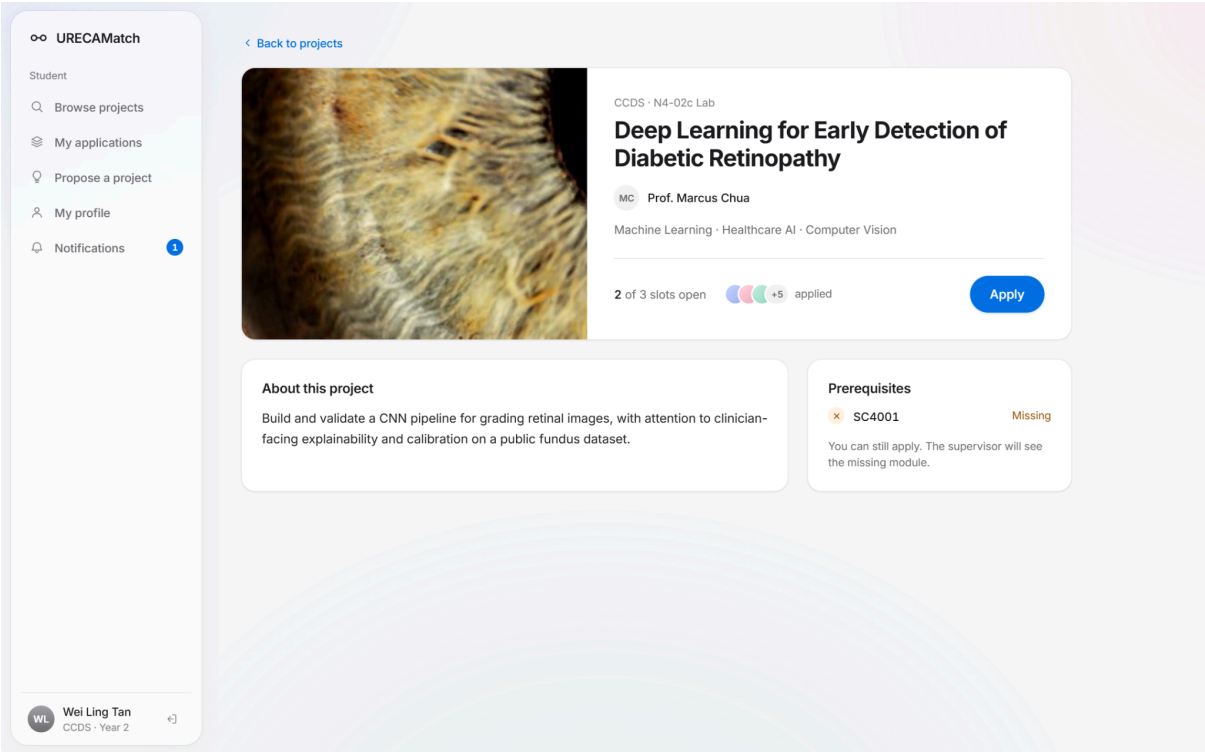


Figure 3.14 ProjectDetailPage

Full description, prerequisites checked against the student's completed modules, slots and the Apply action.

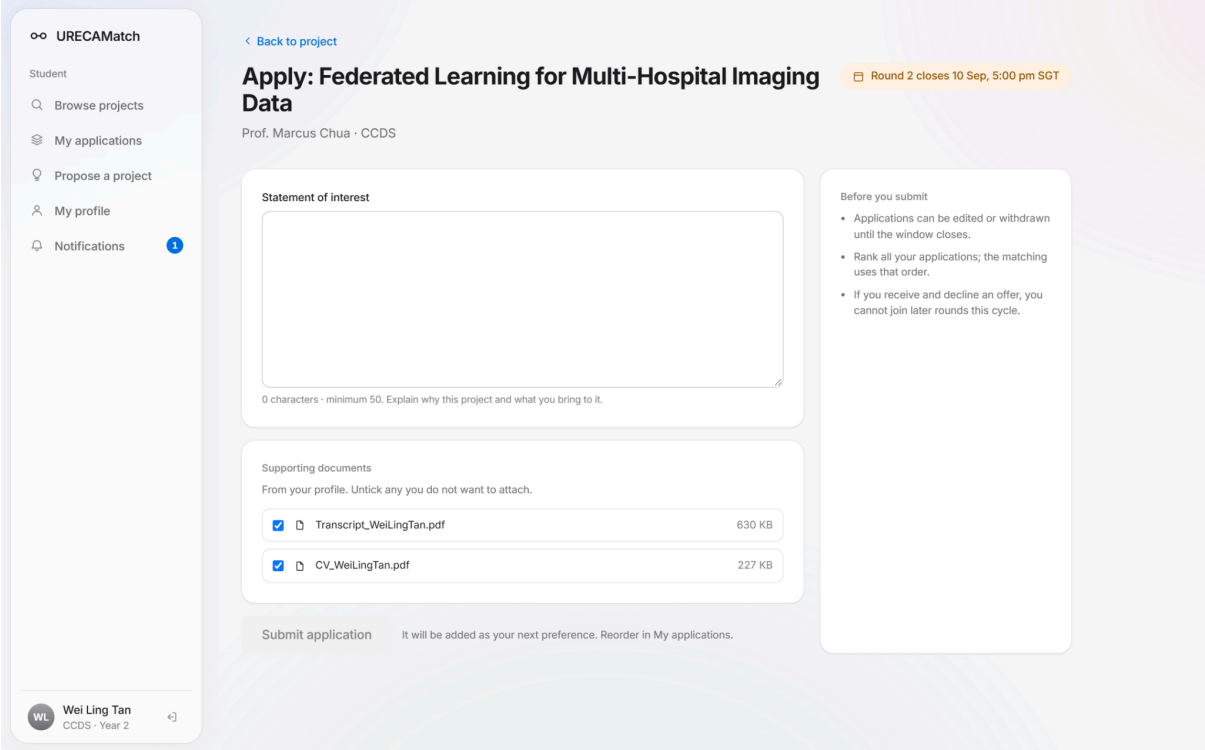


Figure 3.15 ApplyProjectPage

Statement of interest and documents attached from the profile, with the rules that apply.

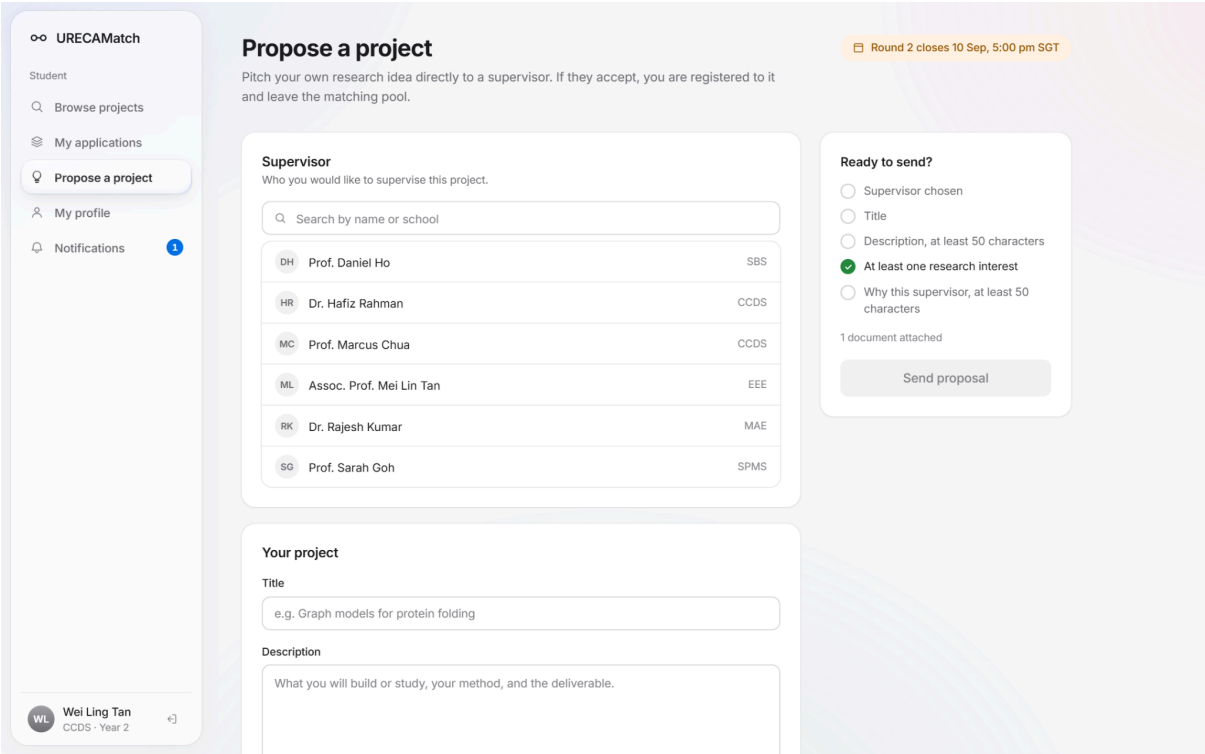


Figure 3.16 ProposeProjectPage

Searchable supervisor picker, title, description, research interests and statement, with a checklist before sending.

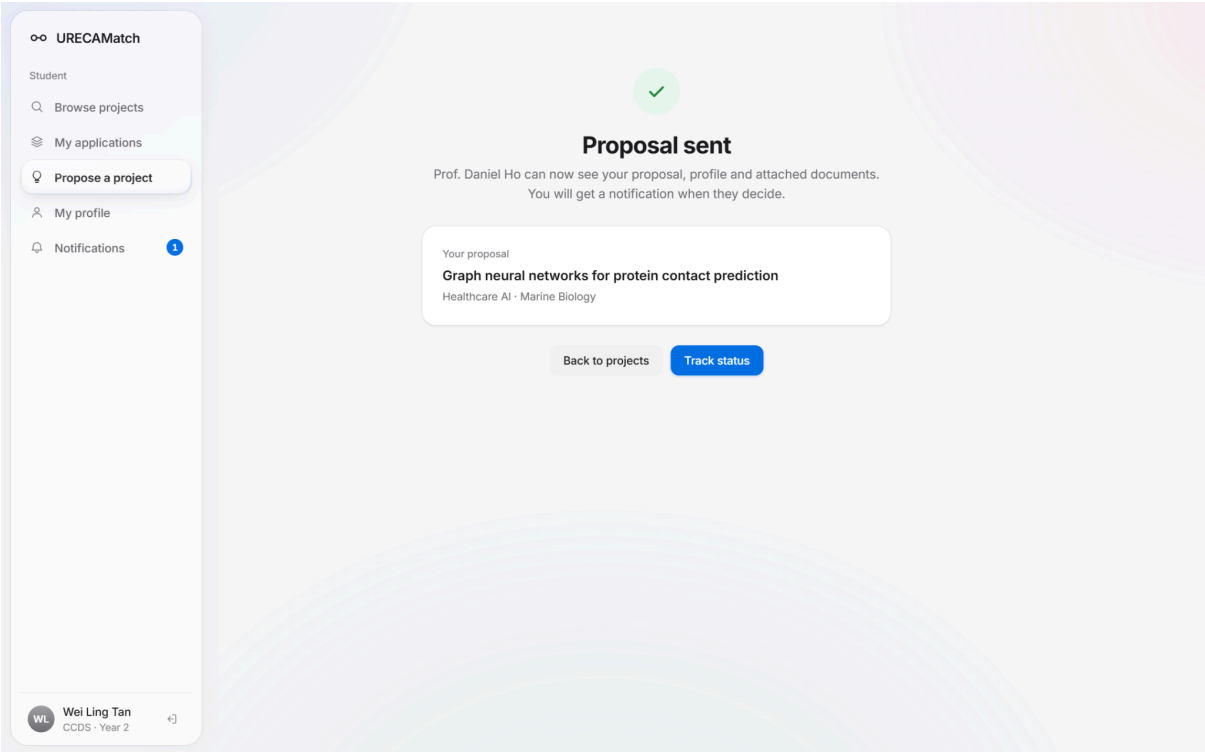


Figure 3.17 ProposalSuccessPage

Confirmation that the proposal was sent, naming the supervisor, with links back to projects and to track its status.

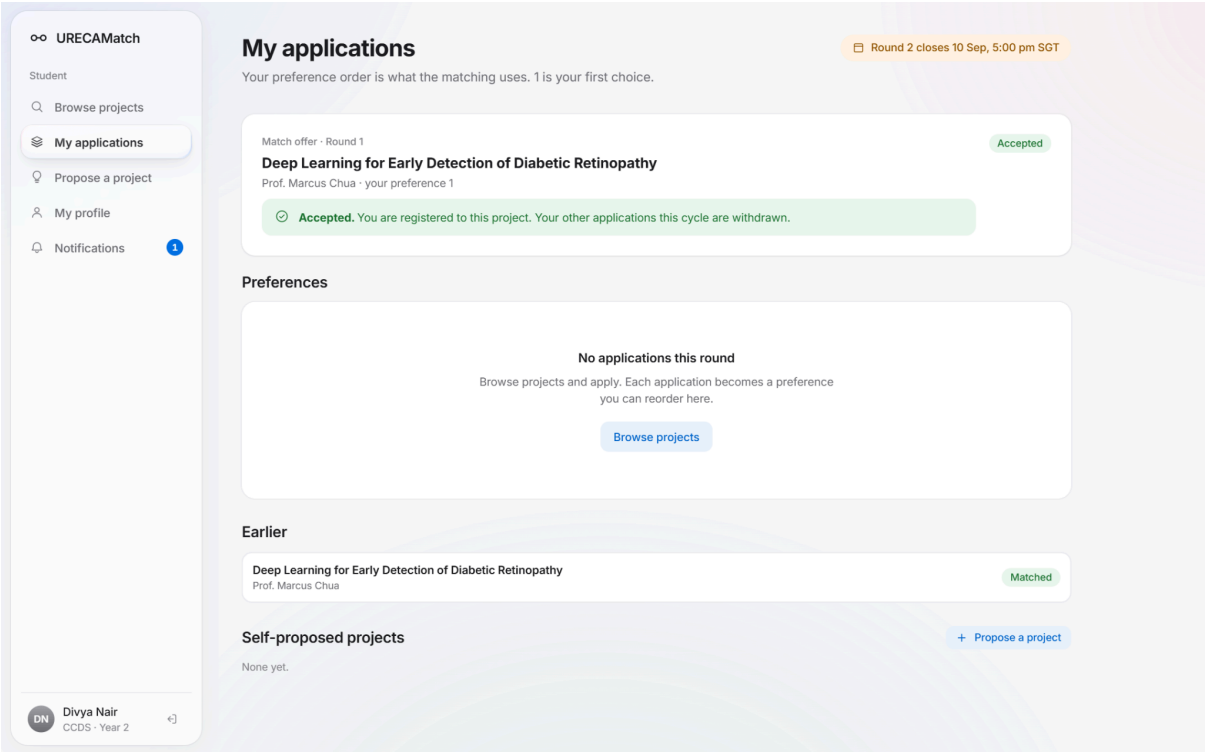


Figure 3.18 MyApplicationPage

Match offer with Accept and Decline, preference ranking for the current round, earlier rounds and self-proposed projects. Shown for a student who has accepted an offer.

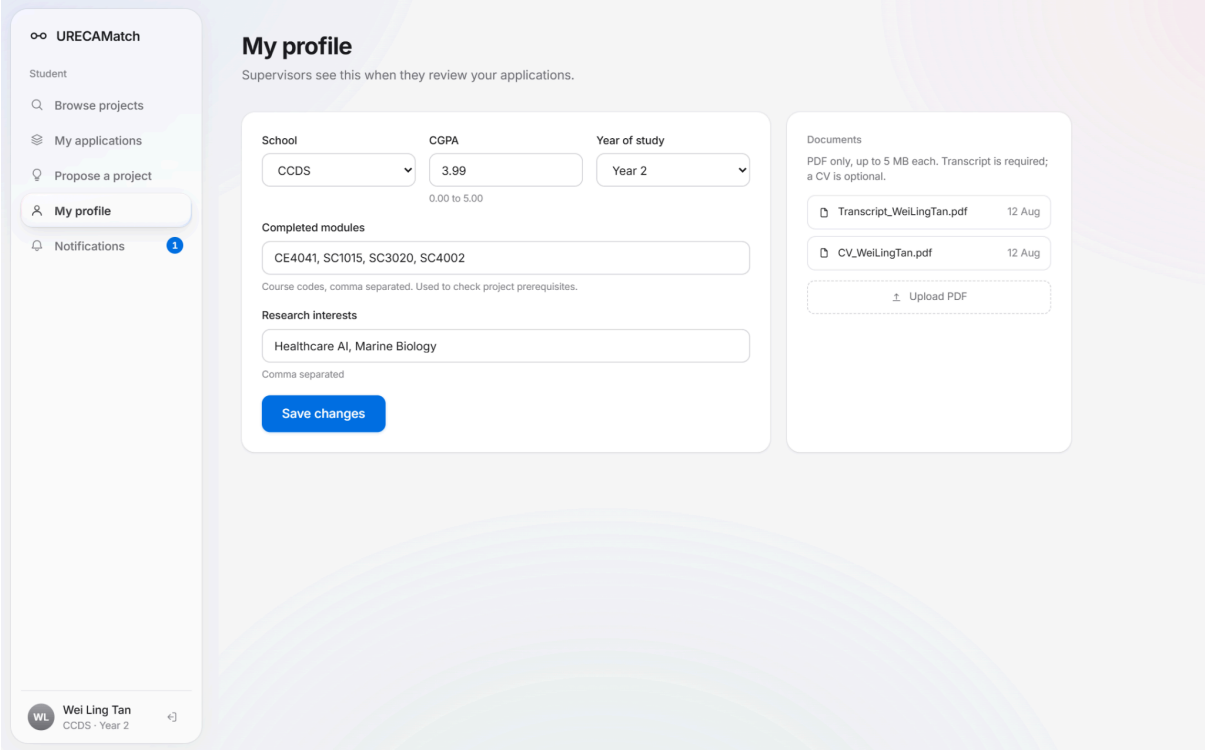


Figure 3.19 MyProfilePage

Academic details and uploaded documents.

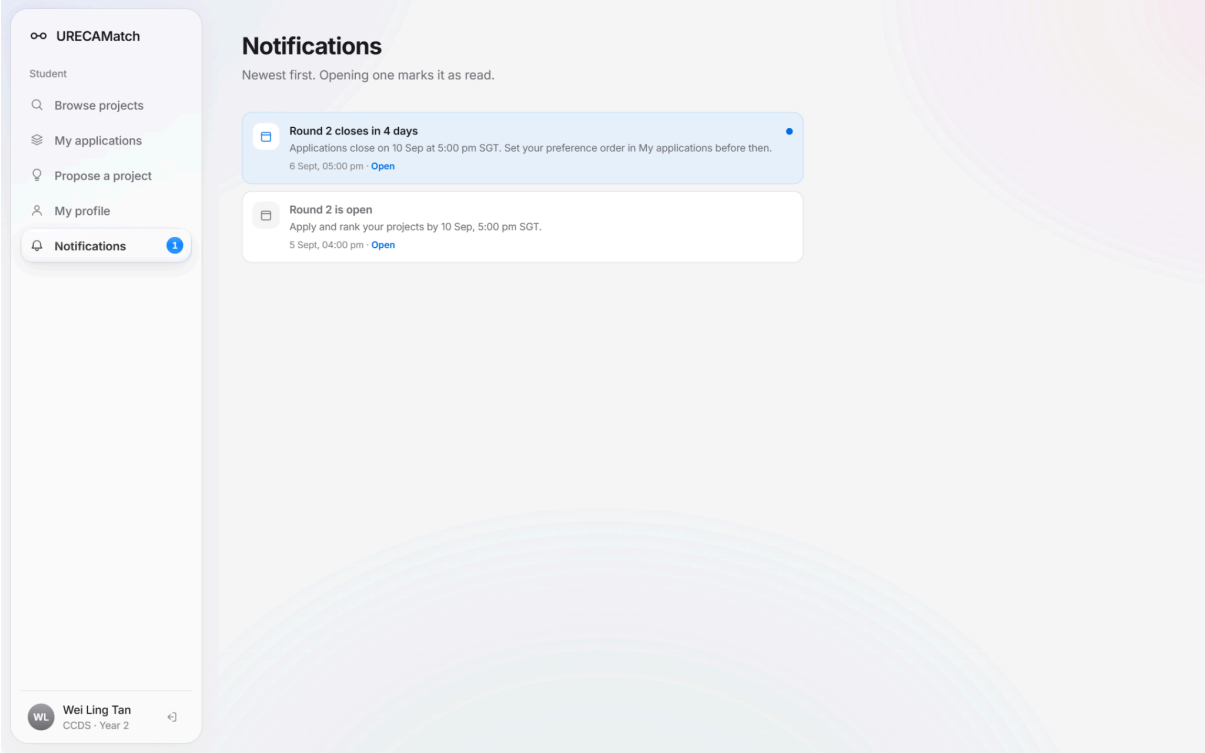


Figure 3.20 NotificationPage

Notifications, newest first. Unread items are highlighted; opening one marks it read.

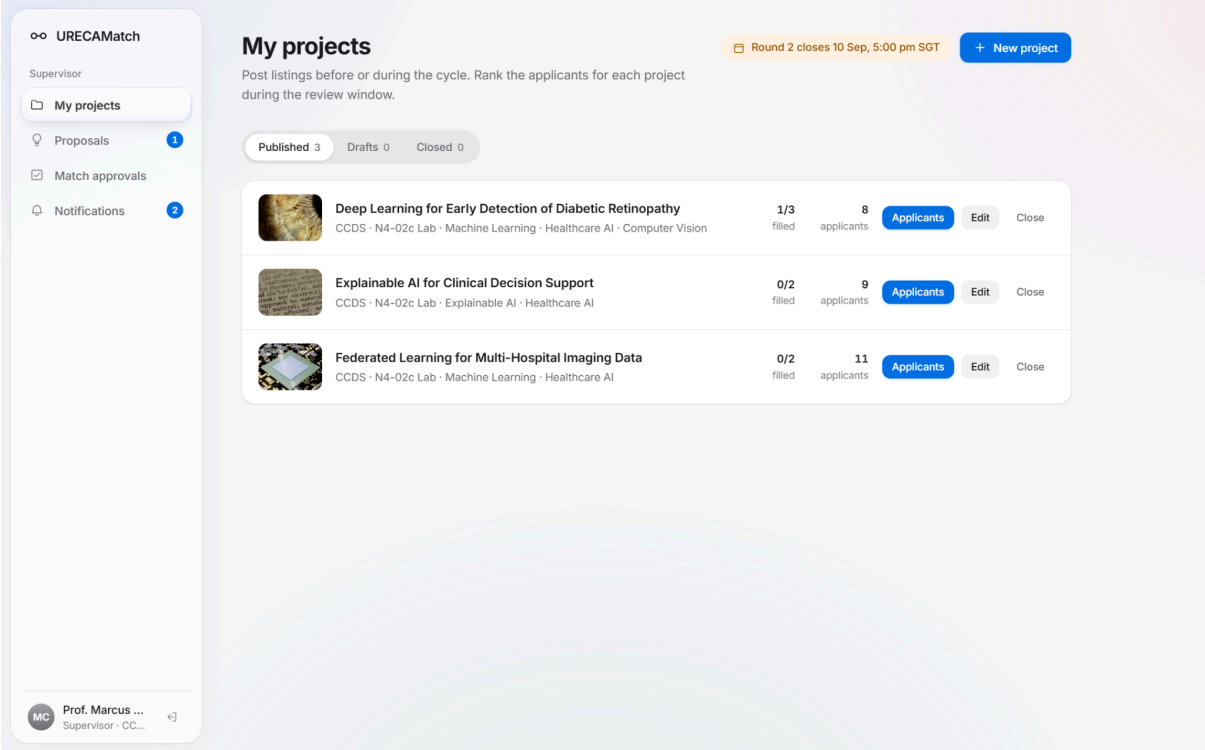


Figure 3.21 SupervisorProjectsPage

Published, Drafts and Closed tabs, with filled slots, applicant counts and the Applicants, Edit and Close actions.

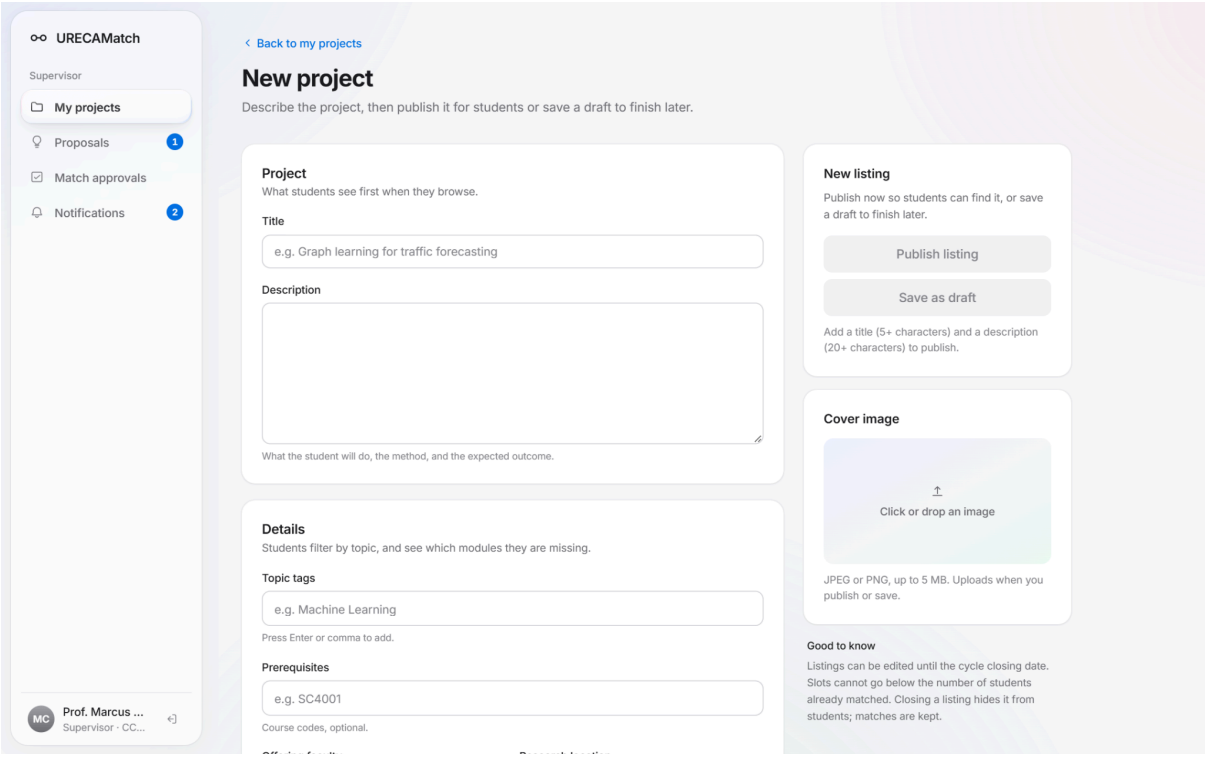


Figure 3.22 CreateProjectPage

Title, description, topic tags, prerequisites, slots, location and cover image. Publish or save as a draft.

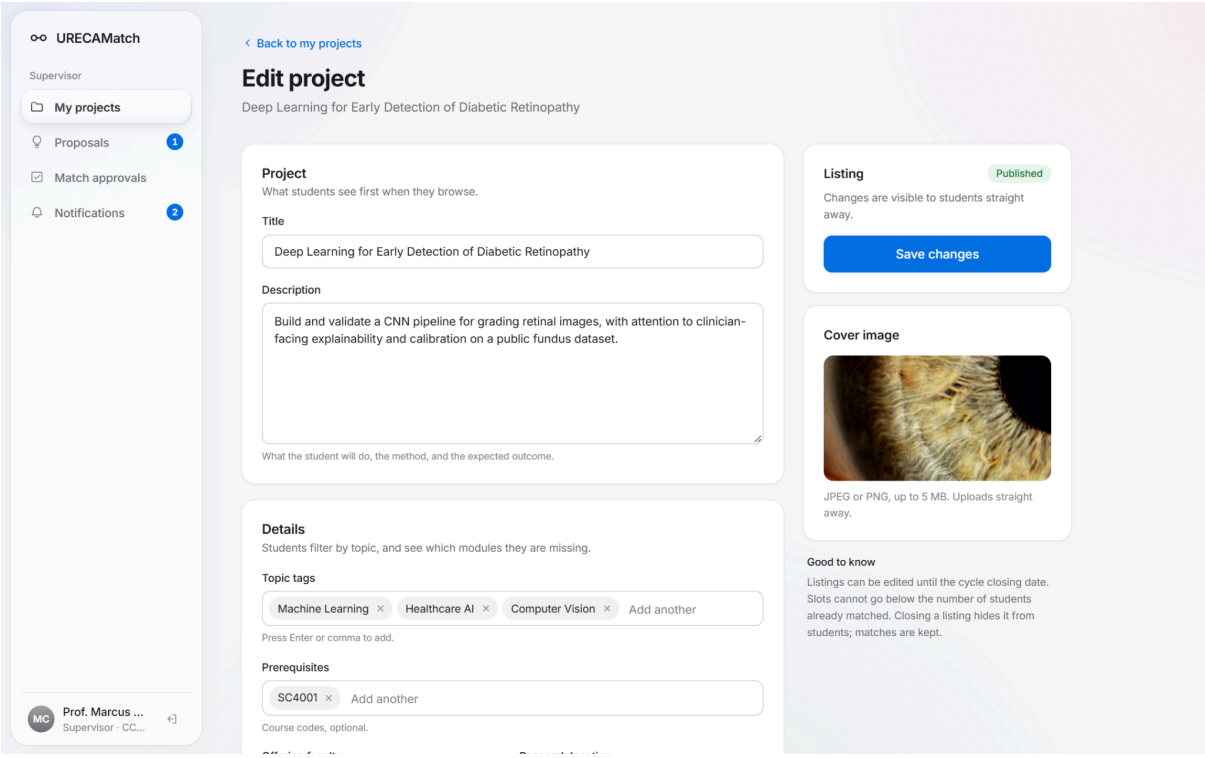


Figure 3.23 EditProjectPage

The same fields as CreateProjectPage, prefilled. Slots cannot go below the number already filled, and editing closes with the cycle.

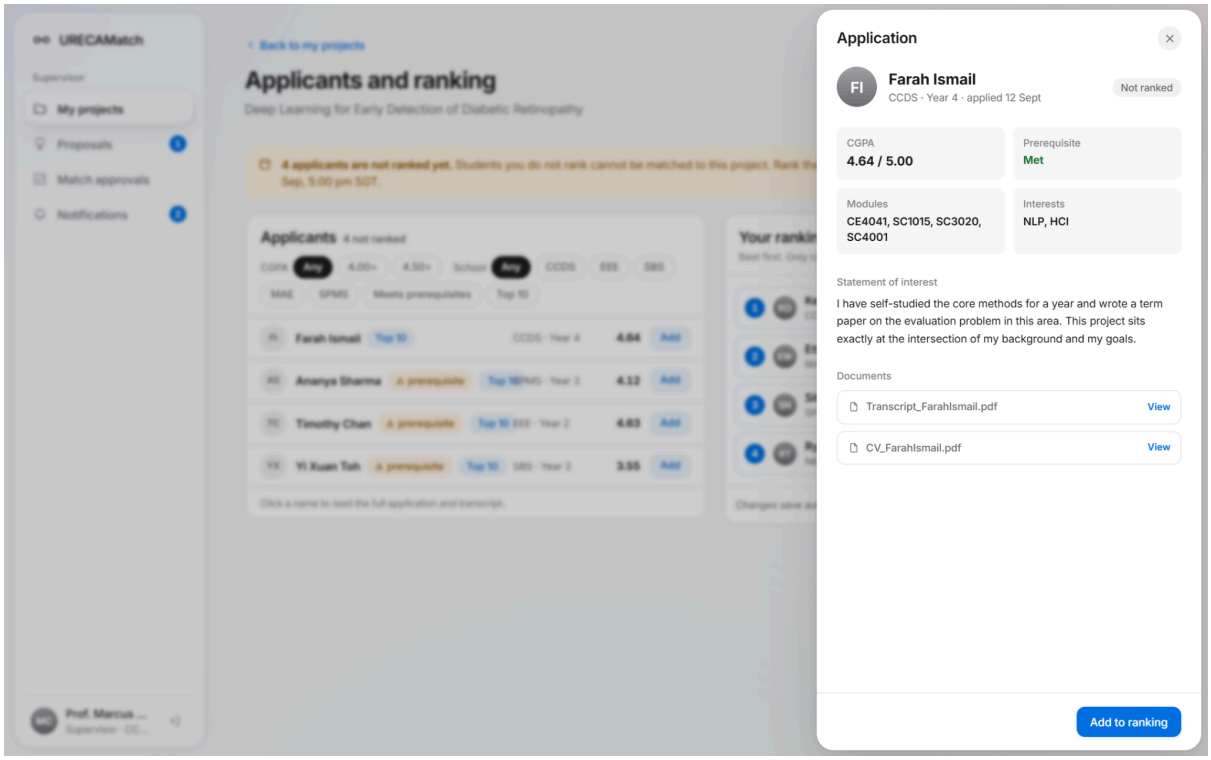


Figure 3.24 ApplicantAndRankingsPage

Applicants filtered by CGPA, school, prerequisites and Top 10, and the supervisor's ranking. The drawer shows one application and its documents.

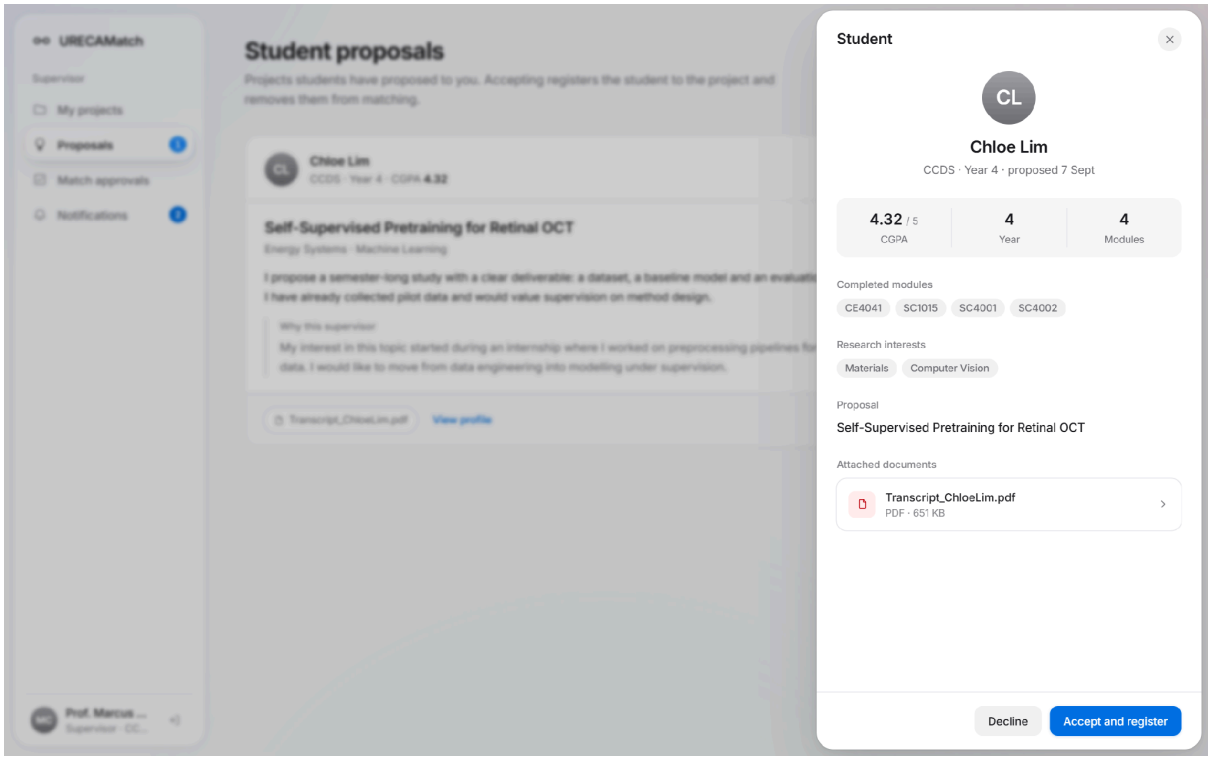


Figure 3.25 StudentProposalsPage

Incoming proposals. The drawer shows the student's profile and attached documents.

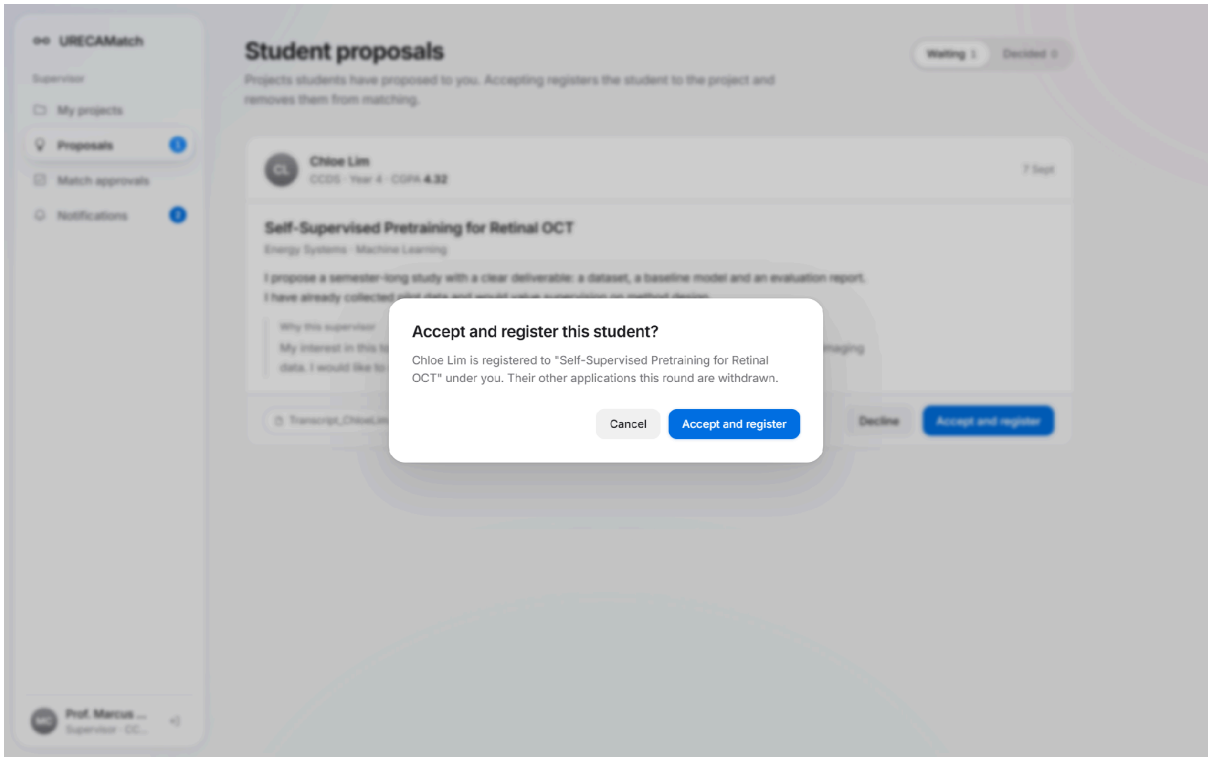


Figure 3.26 StudentProposalsPage, confirmation

Confirmation modal before accepting a proposal, stating its consequence.

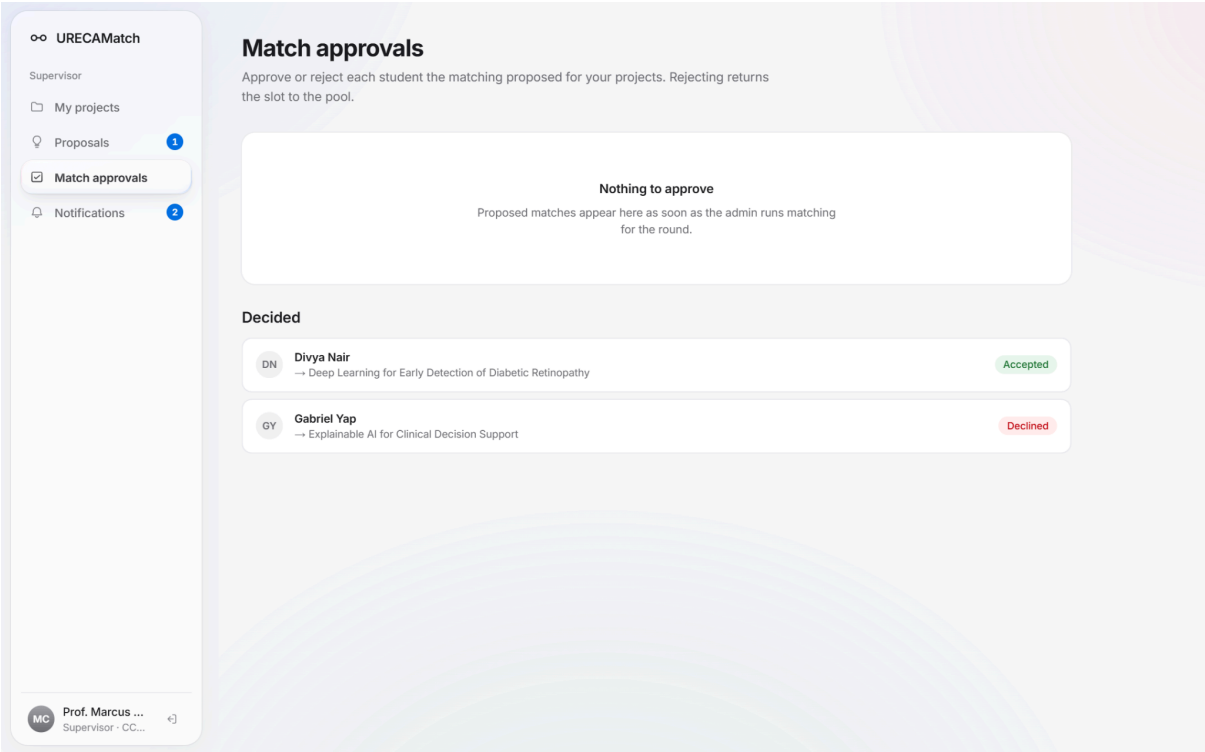


Figure 3.27 MatchApprovalsPage

Matches already decided; none are pending on the demo date. When matching has run, each proposed match can be approved or rejected, View profile opens the student's application in a drawer, and rejecting asks for a reason.

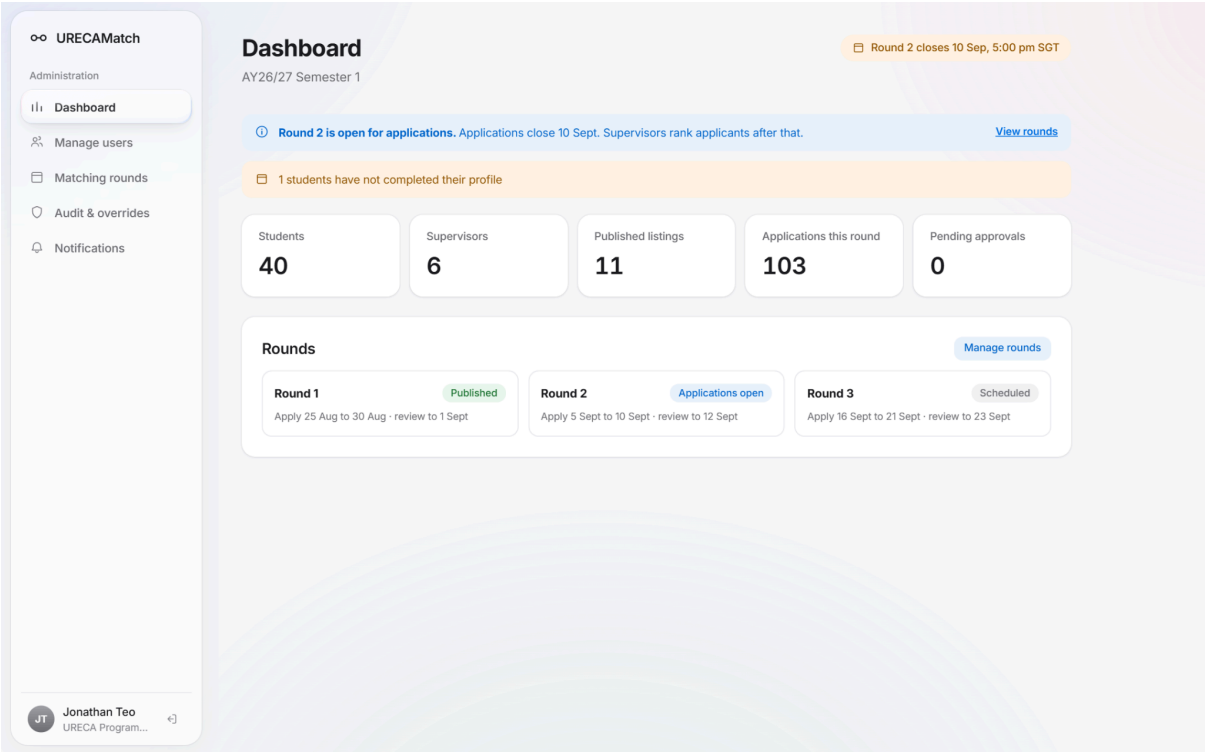


Figure 3.28 AdminDashboardPage

Current round, alerts and counts of students, supervisors, listings, applications and pending approvals.

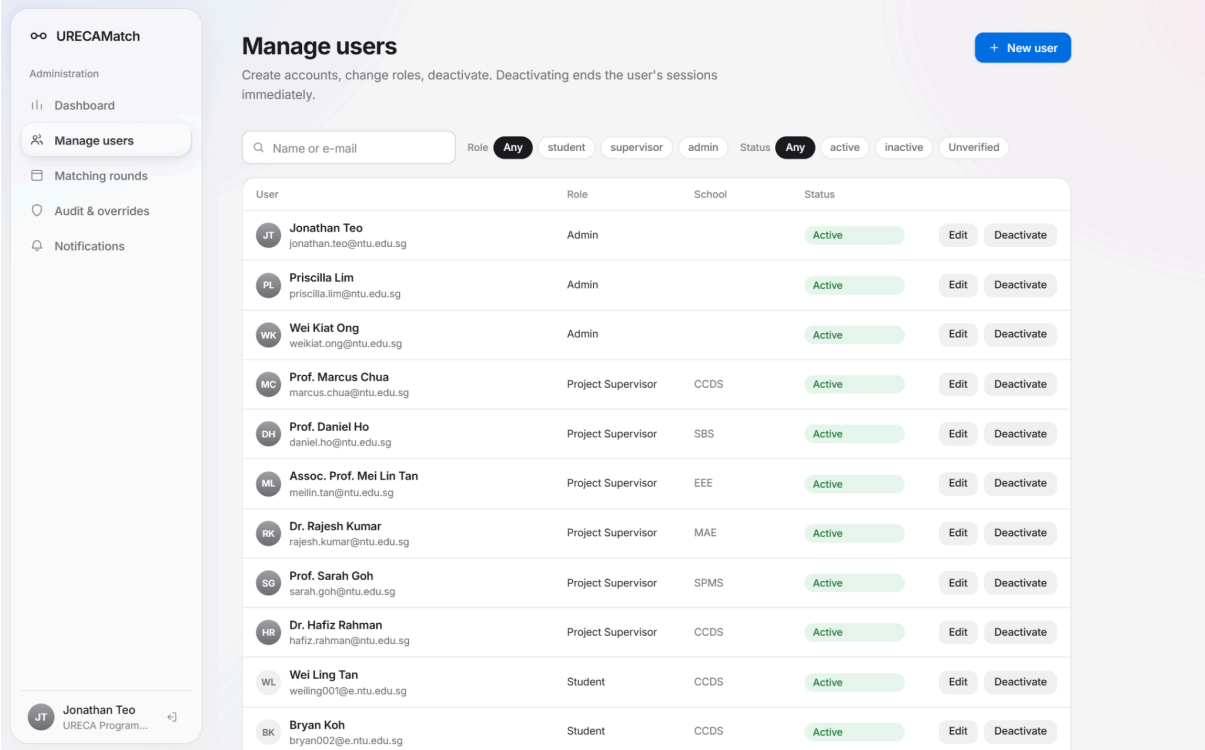


Figure 3.29 ManageUsersPage

Search and filter by role and status; create, edit and deactivate accounts.

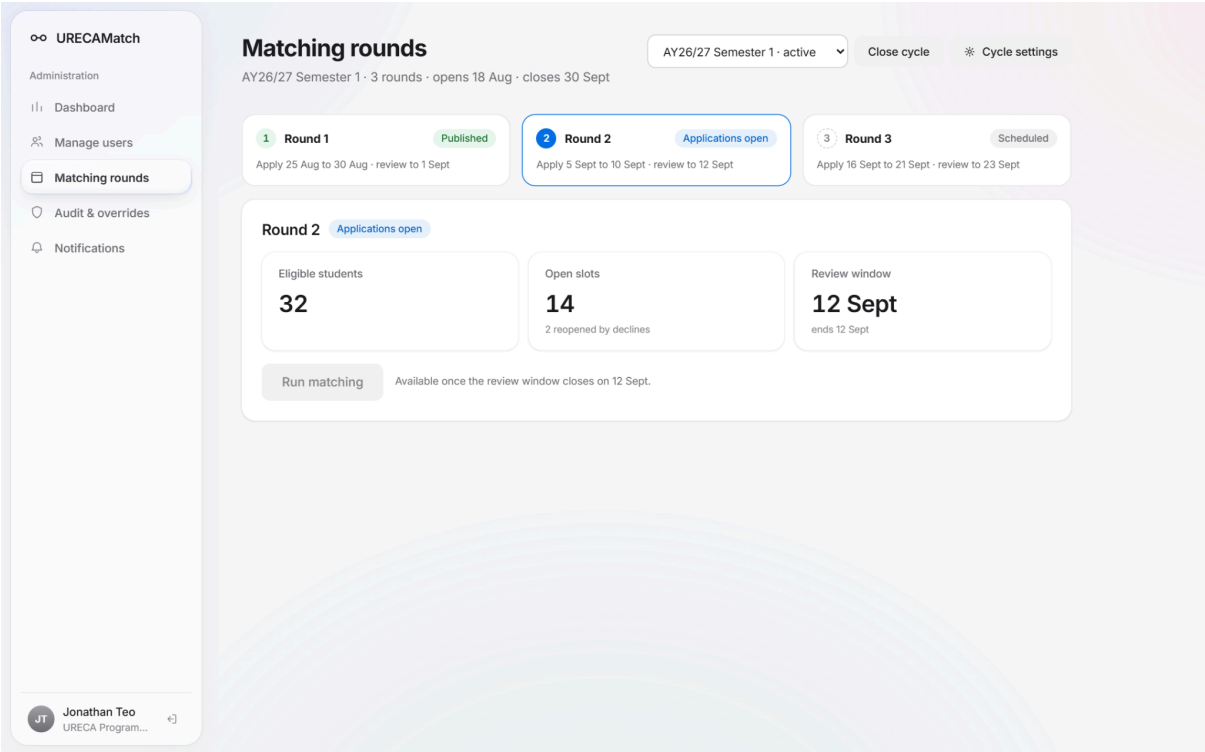


Figure 3.30 MatchingRoundsPage

Cycle selector and settings, the round timeline, the current round summary and Run matching. Round settings take window lengths, optional start dates and the rollover choice.

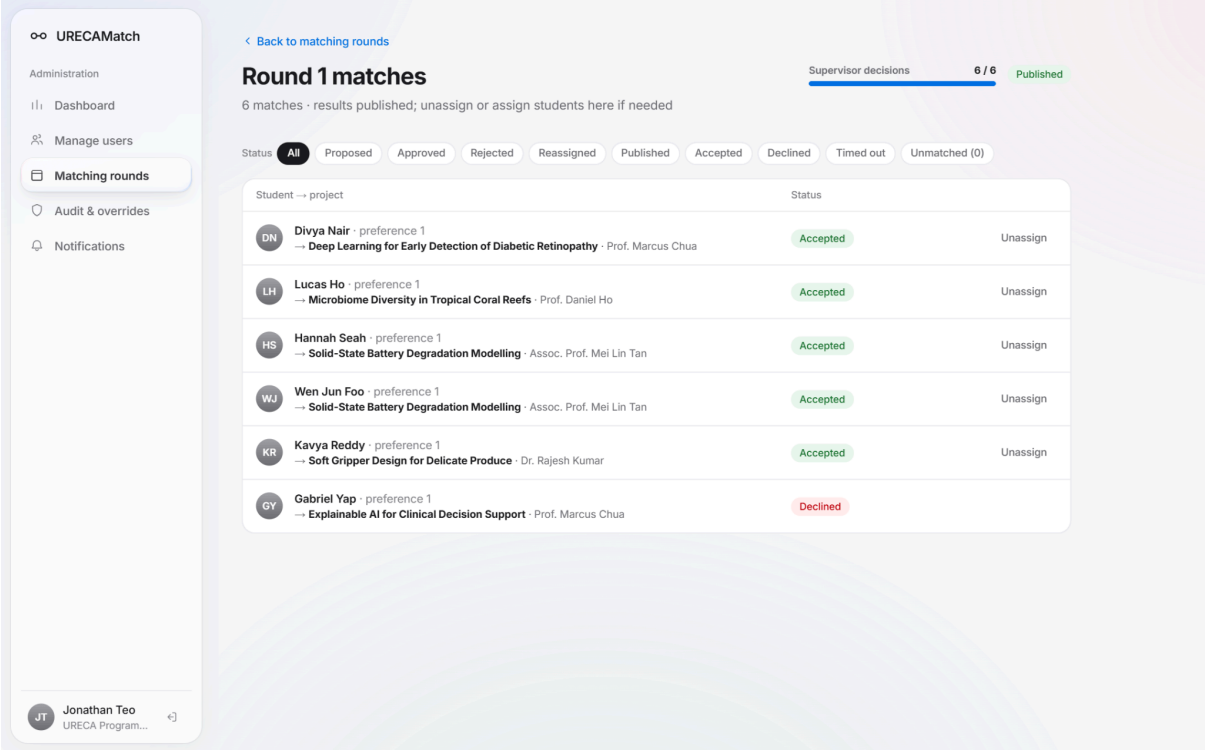


Figure 3.31 MatchingAwaitPublishPage

Matches of one round filtered by status, the Unmatched list with Manual assign, and reassignment, unassignment and publication.

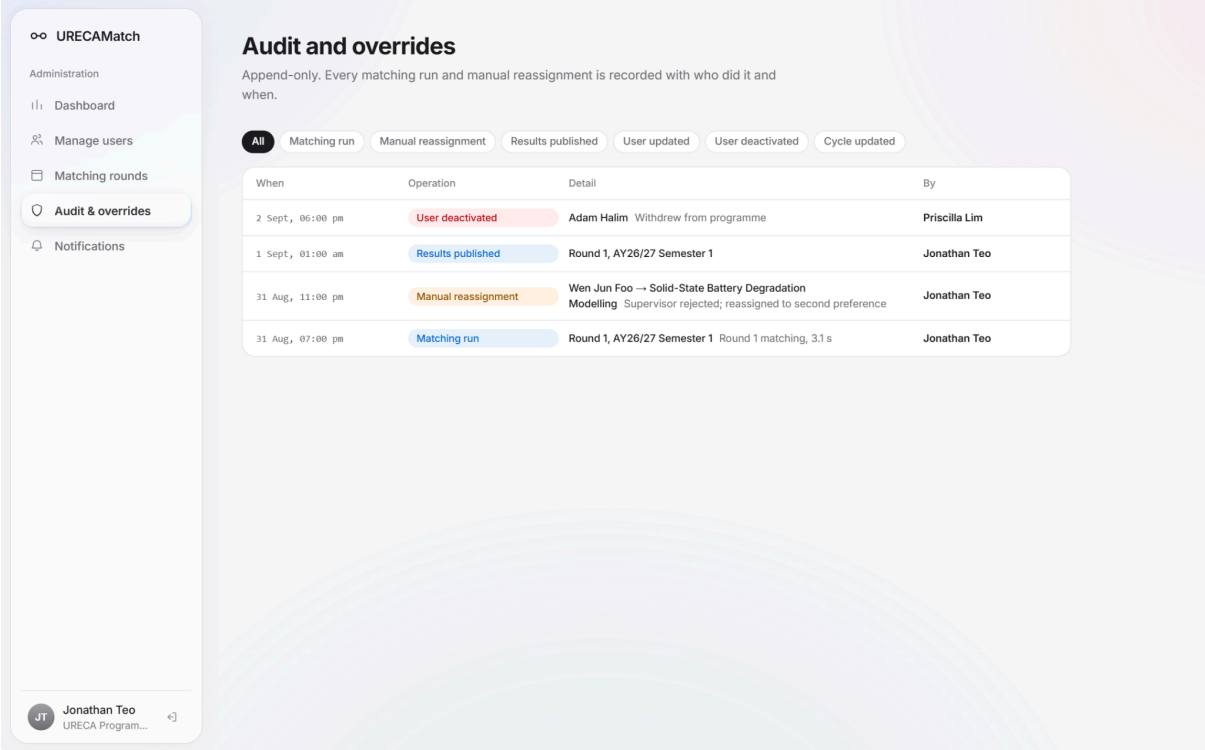


Figure 3.32 AuditOverridesPage

Append-only log of administrative operations, filtered by operation.

3.2 Hardware Interfaces

URECAMatch requires no dedicated hardware. It runs in a web browser on the user's own computer and on cloud servers operated by its hosting providers (3.3), and it makes no use of a camera, microphone, location sensor, biometric sensor or printer.

3.2.1 Supported Devices

The system supports desktop and laptop computers running a current version of Chrome, Edge, Safari or Firefox, in a browser window at least 1200 px wide, with a keyboard and a mouse or touchpad. Tablets and smartphones are not supported in this release. The sidebar's refracting glass effect relies on a browser filter that Chrome and Edge support; Safari and Firefox show a plain frosted panel instead, and every function works the same in all four browsers.

3.2.2 Device Storage

The system reads a file from the user's device only when the user selects it in the browser's file picker. Students upload supporting documents, such as academic transcripts, as PDF files; Project Supervisors upload listing cover images as JPEG or PNG files. Each file may be at most 5 MB, and any other file is rejected with a message giving the reason. Documents are viewed inside the browser, so no other software is needed, and nothing is written to the device unless the user chooses to download a file.

3.3 Software Interfaces

URECAMatch consists of two applications built by the team, a web client and an API server, and depends on five external services (Figure 3.33). The web client presents the interface and holds no data of its own. The API server holds every business rule, checks the user's role on every request, and is the only component that reads or writes data. The two applications share one contract, an OpenAPI document generated from the API, from which the web client's data types are also generated, so that both always agree on every request and response.

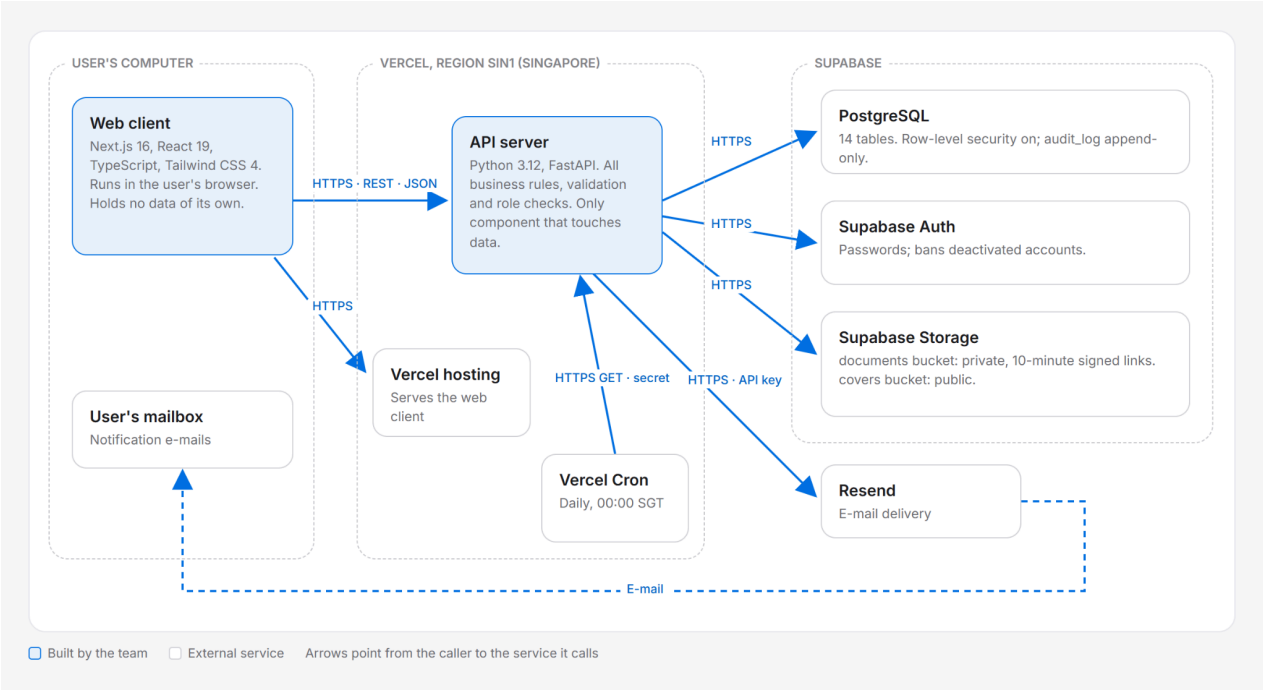


Figure 3.33 Software interfaces

3.3.1 Components and Data Exchanged

Table 3.2 lists each component with its product and version, its purpose and the data exchanged with it.

Table 3.2 Software components and the data they exchange

Component	Product and version	Purpose	Data exchanged
Web client	Next.js 16.3, React 19.2, TypeScript 5, Tailwind CSS 4	Screens and navigation	Calls the API with JSON defined by the API contract
API server	Python 3.12, FastAPI 0.115 or later, Pydantic 2	Business rules, validation, access control by role	All records, through the Store interface
API contract	OpenAPI 3.1 (shared/openapi.json)	One definition of every endpoint and data shape	Web client types are generated from it
Database	PostgreSQL, managed by Supabase	Persistent records	14 tables, from users to audit_log; the audit log is append-only

Component	Product and version	Purpose	Data exchanged
Authentication	Supabase Auth	Passwords and account bans	E-mail address, password, ban on deactivation
File storage	Supabase Storage	Documents and cover images	Private documents bucket with 10-minute signed links; public covers bucket
E-mail	Resend API, Python SDK 2.x	Notification e-mails	Recipient, subject and body, in batches of up to 100
Hosting	Vercel, region sin1 (Singapore)	Runs both applications; Vercel Cron calls the daily update	One request a day to /internal/tick

3.3.2 Document Storage

Supporting documents contain personal data and are therefore never publicly accessible (Figure 3.34). When a Student uploads a document, the API checks its type and size, stores the file in a private storage bucket under the Student's own folder, and records its name, size and location in the database. When a user asks to view a document, the API first confirms that the user is the owner, an Admin, or a Supervisor of a listing the Student applied to, and only then creates a signed link that expires after 10 minutes. The browser uses that link to show the document in the in-app viewer. Listing cover images contain no personal data and are served from a separate public bucket.

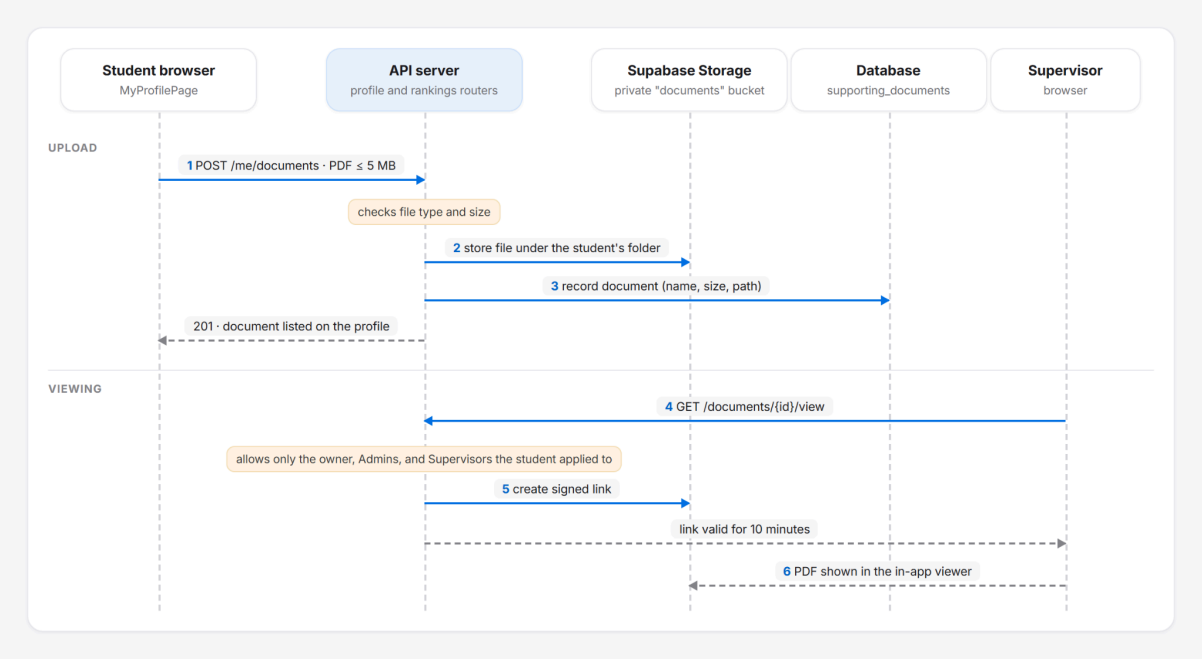


Figure 3.34 Document upload and viewing

3.3.3 Notifications and E-mail

Every notification is created by a single notification service, which no other part of the system bypasses (Figure 3.35). When an event that a user must know about occurs, such as published results, proposed matches awaiting approval, a decision on a proposal or a change to the cycle

dates, the service stores one notification for each recipient and then passes them to the e-mail service, which sends them through Resend in batches of up to 100 within 15 minutes (NFR 5.1.2). The two channels are independent: if e-mail delivery fails, the failure is logged and the in-app notification remains, so that no notice is lost.

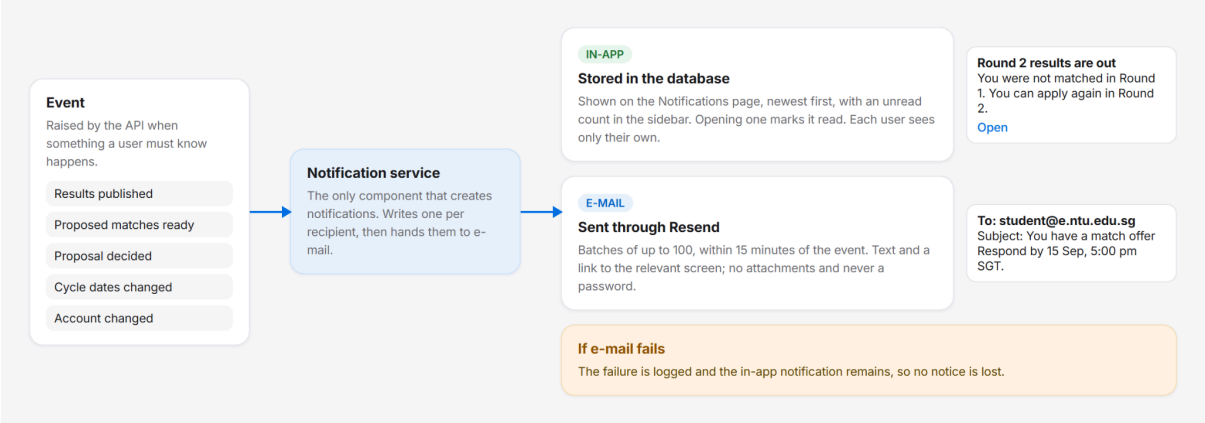


Figure 3.35 Notification flow

3.3.4 Design Constraints

Every software interface follows the same constraints. The API reaches all data through one Store interface, which has one implementation backed by Supabase and one held in memory for development, automated tests and demonstrations; the web client likewise has a live mode and a mock mode, and configuration selects between them. Row-level security is enabled on every database table, so that browsers cannot read the database directly, and a database trigger rejects every change to or deletion of an audit log entry, including changes attempted by the API. The API contract and the web client's types are generated from the code, never edited by hand, and checked for differences on every build. Keys for Supabase, Resend and the scheduled job are held in server configuration and are never sent to the browser.

3.4 Communications Interfaces

All communication between the components of URECAMatch, and between the system and its users, uses HTTPS with TLS encryption. Dates and times are exchanged in ISO 8601 format in UTC and converted to Singapore time only for display.

3.4.1 Communication Channels

Table 3.3 lists each channel with its protocol, format and authentication.

Table 3.3 Communication channels

Channel	Protocol and format	Authentication	Notes
Browser to web client	HTTPS on port 443; HTTP on port 80 redirects to HTTPS	Not required	Delivers the application
Web client to API	REST over HTTPS; JSON in UTF-8; files as multipart/form-data	Session token in an Authorization: Bearer header	Accepted only from the URECAMatch web addresses (CORS)

Channel	Protocol and format	Authentication	Notes
API to Supabase	HTTPS through the Supabase client	Service key, server only	Database, authentication and file storage
API to Resend	HTTPS REST	API key, server only	Outgoing e-mail
Vercel Cron to API	HTTPS GET, daily at 00:00 Singapore time	Shared secret	Advances round status; times out unanswered offers
E-mail to users	Sent through Resend	Not applicable	Within 15 minutes of the event (NFR 5.1.2); text and a link, no attachments, never a password

3.4.2 Message Format, Sessions and Errors

The web client and the API exchange JSON request and response bodies in UTF-8, as defined by the API contract (Figure 3.36). Signing in returns a session token, which the web client sends with every later request. A session ends after 30 minutes without activity (REQ-AUTH-3), after which the API rejects the token and the user is returned to the sign-in page. A new account is activated with a 6-digit verification code sent to the user's NTU e-mail address, which expires after 10 minutes and can be requested again (REQ-REG-2, REQ-REG-4).

Every failed request returns an HTTP status and a JSON body containing an error code, a plain-English message and, where the problem lies in one input, the name of that field. The status identifies the kind of failure: 401 when the user is not signed in or the session has expired, 403 when the action is not allowed for the user's role or the account is inactive, 404 when a record does not exist or is not visible to the user, 409 when the request conflicts with the current state, such as applying twice to the same project, and 422 when an input is invalid. The web client shows the message to the user (3.1.6), while the error code allows the program to respond to the failure.

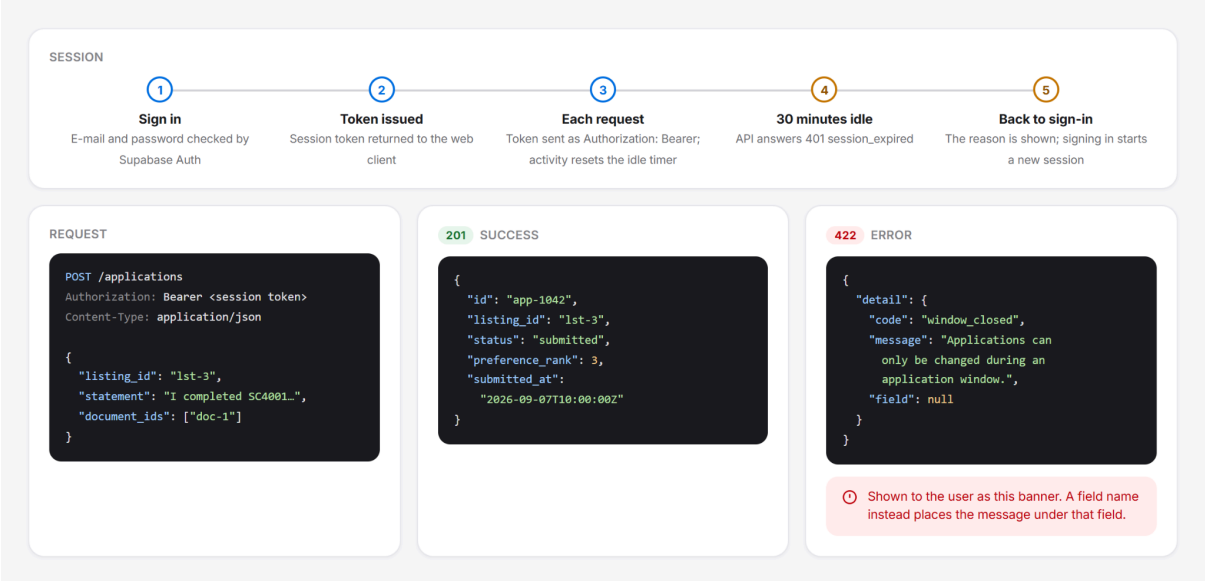


Figure 3.36 Requests, sessions and errors

3.4.3 Security and Performance

Passwords are checked by Supabase Auth and are never stored or handled by URECAMatch itself, and passwords and keys are never included in e-mail. Each channel to an external service is authenticated with a key held only on the server. Uploaded files are limited to 5 MB, and responses are designed to meet the responsiveness targets in NFR 5.1. The web client does not receive pushed updates; it fetches current data, including the unread notification count, each time the user opens or refreshes a screen.

4. System Features

This section describes the system features of URECAMatch, grouped by the user role that initiates them: Admin (4.1 to 4.8), Student (4.9 to 4.14), Project Supervisor (4.15 to 4.18) and all users (4.19 to 4.21). For each feature it gives a description and priority, the stimulus/response sequences, and the functional requirements.

Each functional requirement has a unique identifier of the form REQ-AREA-n. The identifier is used to trace the requirement to its use case, screen (Table 3.9), code and test. High priority means the feature is needed for a matching cycle to run; Medium means it is important but a cycle can run without it.

Admin Features (4.1 to 4.8)

4.1 ManageApplicationCycle

4.1.1 Description and Priority

Description

This feature enables an Admin to create a new Application Cycle or modify an existing one, configuring time parameters such as start and end dates, and to close or reopen a cycle.

Priority

High; plays a critical role in setting up and adjusting the timeline for URECAMatch application periods.

4.1.2 Stimulus/Response Sequences

1. After the Admin successfully signs in, the Admin navigates to Matching Rounds and creates a new cycle, or opens Cycle settings for the selected cycle.
    a) Admin selects the option to create a new application cycle.
        i) The System displays the Application Cycle configuration where the Admin sets the academic year, the application opening and closing dates, and the pre-application viewing period.
        ii) The System validates the provided parameters and saves the newly created Application Cycle upon confirmation.

b) Admin selects the option to modify an existing application cycle.
   i) The System displays the configuration details of the selected application cycle.
   ii) The System allows the Admin to change the start and end dates for the active or upcoming cycle before those dates have passed.

c) Admin selects the option to close the active cycle, or to reopen a closed cycle.
   i) The System confirms the action, updates the cycle status and notifies all Students and Project Supervisors.

4.1.3 Functional Requirements

- **REQ-CYCLE-1** The system shall allow an Admin to create an application cycle by setting its academic year, opening date, closing date and pre-application viewing period (0 to 60 days). The system shall reject the cycle if its closing date is not after its opening date, if its dates overlap another cycle, or if another cycle is still active.
- **REQ-CYCLE-2** The system shall allow an Admin to modify an existing application cycle's start and end dates before those dates have passed. Matching Rounds that have not started shall move with a changed opening date and keep their spacing.
- **REQ-CYCLE-3** When the closing date of an active application cycle is modified, the system shall send a notification to all Students and Project Supervisors.
- **REQ-CYCLE-4** The system shall allow an Admin to close the active cycle and to reopen a closed cycle when no other cycle is active. Closed cycles and their results shall be kept. All Students and Project Supervisors shall be notified.

4.2 ManageMatchingRounds

4.2.1 Description and Priority

Description

This feature enables an Admin to configure multiple sequential matching rounds within the active application cycle, each consisting of an Application Window followed by a Review Window. The duration of each window is configurable by the Admin.

Priority

High; essential for defining the structured timeline of multi-round student applications and supervisor rankings.

4.2.2 Stimulus/Response Sequences

1. After the Admin navigates to the Application Cycle configuration, the Admin selects the option to manage matching rounds.
   a) Admin selects the option to add or configure matching rounds.
      i) The System generates the interface for up to three sequential matching rounds, where the Admin can set each round's optional start date and the length of its application and review windows.

b) Admin selects the option to modify existing matching rounds.
   i) The System displays the previously configured matching round dates for editing. Rounds that have already started cannot be changed.
   ii) The System validates that none of the rounds overlap with each other and that all rounds fall within the global Application Cycle start and end dates.

4.2.3 Functional Requirements

- **REQ-ROUND-1** The system shall allow an Admin to configure 1 to 3 sequential Matching Rounds in an Application Cycle. For each round the Admin shall set the Application Window length (1 to 14 days), the Review Window length (1 to 7 days) and, optionally, a start date. A round without a start date shall begin when the pre-application period ends (Round 1) or 4 days after the previous round's Review Window ends.
- **REQ-ROUND-2** The system shall allow an Admin to change or remove a Matching Round only before its Application Window starts.
- **REQ-ROUND-3** The system shall reject the configuration if the calculated dates for a matching round overlap with another matching round or fall outside the global Application Cycle dates.

4.3 ExecuteMatchingProcess

4.3.1 Description and Priority

Description

This feature enables an Admin to execute the stable-matching algorithm to assign students to projects based on ranked preferences and supervisor rankings.

Priority

High; essential for processing allocations and rolling over unmatched entities.

4.3.2 Stimulus/Response Sequences

1. The Admin navigates to the Matching Rounds page and selects "Run matching".
2. The System verifies Student eligibility, excluding any Student who already holds a match this cycle or who declined or let an offer time out in an earlier round.
3. The System runs the matching algorithm against rankings and verifies capacity constraints.
4. The System commits match results to storage and logs an immutable entry with Admin ID and timestamp.
5. The System notifies each Project Supervisor who has a proposed match. Rollover of unmatched Students happens when results are published (4.5).

4.3.3 Functional Requirements

- **REQ-EXEC-1** The system shall only permit the matching process to be initiated after the round review window has passed.

- **REQ-EXEC-2** The system shall prevent the matching process from being initiated more than once per matching round in the application cycle.
- **REQ-EXEC-3** The system shall allow an Admin, when configuring rounds, to choose whether the applications of unmatched eligible Students roll over into the next round. Open slots of published listings, including slots released by rejected, declined or timed-out matches, shall carry over to the next round automatically.
- **REQ-EXEC-4** The matching process shall use student-proposing deferred acceptance with listing capacities. It shall exclude Students who are ineligible or already hold a match this cycle, use only slots no other match holds, and match a Student to a listing only if its Project Supervisor ranked that Student.
- **REQ-EXEC-5** After execution, the system shall set every resulting match to Proposed and notify each affected Project Supervisor.

4.4 ReviewMatchingResults

4.4.1 Description and Priority

Description

This feature enables an Admin to verify supervisor approvals, handle manual reassignments, and publish finalised matching outcomes to students and supervisors.

Priority

High; critical for finalising and releasing match allocations.

4.4.2 Stimulus/Response Sequences

1. The Admin navigates to the "Matching Round" page.
2. The System displays the proposed matches along with supervisor approval statuses.
3. The Admin confirms that matches have been approved by the respective Project Supervisors.
4. Where needed, the Admin reassigns a match to another project with an open slot and enters a justification.

4.4.3 Functional Requirements

- **REQ-REV-1** The system shall allow an Admin to filter a round's matches by status (Proposed, Approved, Rejected, Reassigned, Published, Accepted, Declined, Timed out) and to view the Students who applied in the round but were not matched.
- **REQ-REV-2** The system shall allow an Admin to reassign a match that has not yet been offered to a Student to another published project with an open slot, with a justification of at least 10 characters. If the round is already published, the reassigned match shall be offered to the Student immediately.

4.5 PublishResults

4.5.1 Description and Priority

Description

This feature enables an Admin to publish the approved matching results so that students and project supervisors can view their final allocations.

Priority

High; crucial for releasing the outcomes of a matching round.

4.5.2 Stimulus/Response Sequences

1. The Admin selects the option to publish matching results after supervisor approvals are confirmed.
2. The System transitions matching results to the published state and sets the expiry of each offer.
3. The System triggers automated notifications to affected students and supervisors.
4. If rollover is enabled, the System carries the applications of unmatched eligible Students into the next round.

4.5.3 Functional Requirements

- **REQ-PUB-1** The system shall allow an Admin to publish a round's results only after matching has run and every proposed match has been approved or rejected, and only once per round.
- **REQ-PUB-2** When results are published, the system shall notify all affected users.
- **REQ-PUB-3** On publication, each offer shall expire when the next round's Review Window starts, or at the cycle closing date for the last round.

4.6 ManualAssignStudent

4.6.1 Description and Priority

Description

This feature allows an Admin to manually assign a Student to a project, reserved for exceptional circumstances mandating a manual override of URECA Project assignments.

Priority

High; essential for handling administrative overrides when standard matching requires manual intervention.

4.6.2 Stimulus/Response Sequences

1. Admin reviews the matching round results.
2. Admin clicks on "Manual assign" next to a Student in the Unmatched list.
3. Admin selects an available URECA Project to be assigned to the Student and enters a justification.
4. Admin clicks "Confirm".

4.6.3 Functional Requirements

- **REQ-ASSIGN-1** The system shall allow an Admin to manually assign a Student to a URECA Project as a manual override.
- **REQ-ASSIGN-2** The system shall require the Admin to input a justification of at least 10 characters before executing a manual assignment override.
- **REQ-ASSIGN-3** If a Student is already assigned to a URECA Project, the system shall display a warning message and prevent assignment until the current project is unassigned.
- **REQ-ASSIGN-4** The system shall allow an Admin to unassign a Student from a project with a reason of at least 10 characters, freeing the slot and notifying the Project Supervisor, and the Student if they had been offered the place.

4.7 MaintainAuditLog

4.7.1 Description and Priority

Description

This feature allows an Admin to view immutable system audit records of administrative operations, including matching execution runs and manual allocation overrides.

Priority

Medium; provides traceability and compliance tracking.

4.7.2 Stimulus/Response Sequences

1. The Admin navigates to the "Audit & Overrides" tab.
2. The System displays timestamped records of all administrative operations along with the initiating Admin (Figure 3.25).
3. The Admin filters or inspects specific log entries.

4.7.3 Functional Requirements

- **REQ-AUDIT-1** The system shall record, with timestamp and initiating Admin: creating, changing, closing and reopening a cycle; configuring rounds; executing matching; reassigning, assigning and unassigning Students; publishing results; and creating, updating and deactivating user accounts.
- **REQ-AUDIT-2** The system shall strictly prevent any user, including an Admin, from modifying or deleting audit log entries.

4.8 ManageUserAccounts

4.8.1 Description and Priority

Description

This feature allows an Admin to create, update or deactivate user accounts in the system.

Priority

High; critical for user administration and access control.

4.8.2 Stimulus/Response Sequences

1. The Admin navigates to the "Manage Users" dashboard.
    a) Admin selects the option to manage and update user details.
        i) The System displays the full directory of users with role and status filters.
        ii) The Admin selects a specific user account to edit, updates their name, role (Student, Project Supervisor, Admin) or status, and clicks "Save" to commit the changes and log the action.
    b) Admin selects the option to deactivate a user account.
        i) The Admin selects "Deactivate" next to an active user.
        ii) The System prompts for confirmation, and upon confirmation, marks the user account as inactive and immediately terminates that user's active sessions.

4.8.3 Functional Requirements

- **REQ-USER-1** The system shall allow an Admin to create a new user account and assign a role (Student, Project Supervisor, Admin).
- **REQ-USER-2** The system shall restrict the creation of Admin-role accounts to existing Admins.
- **REQ-USER-3** When a user account is deactivated, the system shall terminate that user's active sessions.

Student Features (4.9 to 4.14)

4.9 QueryUrecaListings

4.9.1 Description and Priority

Description

Allows a Student to search and browse available project listings.

Priority

High.

4.9.2 Stimulus/Response Sequences

1. Student selects "Browse projects" during pre-application or an active matching round.
2. The system displays project listings filterable by keyword, school, topic tag, and supervisor.

4.9.3 Functional Requirements

- **REQ-BROWSE-1** The system shall allow a Student to search and browse project listings before the formal application cycle opens.
- **REQ-BROWSE-2** The system shall allow students to filter and search listings by keyword, school, supervisor name, and topic tags.
- **REQ-BROWSE-3** The system shall exclude closed or unpublished project listings from student search results.

4.10 SubmitUrecaApplication

4.10.1 Description and Priority

Description

Allows a Student to submit applications and preference rankings during a round's Application Window.

Priority

High.

4.10.2 Stimulus/Response Sequences

1. Student views project details and clicks "Apply".
2. Student writes a Statement of Interest, selects documents from their profile to attach, and clicks "Submit application". The application becomes the Student's lowest preference.
3. The system validates the inputs and checks that an Application Window is open.

4.10.3 Functional Requirements

- **REQ-APPLY-1** The system shall prevent a Student from submitting, withdrawing or re-ranking an application outside a round's Application Window.
- **REQ-APPLY-2** The system shall require a Statement of Interest of at least 50 characters and allow the Student to attach their own uploaded documents.
- **REQ-APPLY-3** The system shall reject an application if the Student's profile is incomplete, the Student is ineligible, the listing is not published or has no open slots, or the Student already has an active application to that listing.
- **REQ-APPLY-4** The system shall allow a Student to withdraw only an application still under review, and shall renumber the remaining preference ranks.

4.11 RankAppliedProjects

4.11.1 Description and Priority

Description

This feature allows a Student to rank and organise their submitted URECA project applications in order of preference during the active application window.

Priority

High; crucial for determining student preference hierarchy during the application process.

4.11.2 Stimulus/Response Sequences

1. The Student navigates to their submitted applications dashboard.
2. The System displays the list of currently submitted URECA project applications.
3. The Student assigns or adjusts the preference order for each submitted project.
4. The Student clicks "Save Rankings".
5. The System validates that all active applications have unique, sequential preference numbers assigned.

6. The System records and saves the updated ranking order.

4.11.3 Functional Requirements

- **REQ-SRANK-1** The system shall allow a Student to rank their submitted applications in order of preference.
- **REQ-SRANK-2** The system shall require the ranking to contain every active application exactly once.

4.12 SelfProposeUrecaApplication

4.12.1 Description and Priority

Description

This feature allows Students to self-propose a URECA Project and submit it to their desired Project Supervisor for approval.

Priority

High; provides an alternative pathway for students to pursue personalised research topics.

4.12.2 Stimulus/Response Sequences

1. Student clicks on "Propose a Project".
2. The System displays the Application page for a Self-Proposed URECA Project.
3. Students fill in their proposed URECA Project title, write-up, research interests, supporting documents, and choice of Project Supervisor.
4. Students click "Send proposal".
5. The System validates all required fields and submits the application for Project Supervisor consideration.

4.12.3 Functional Requirements

- **REQ-PROPOSE-1** The system shall allow a Student to create a self-proposed project with a title, a description of at most 500 words, a statement of at most 300 words and at least one research interest.
- **REQ-PROPOSE-2** The system shall allow a Student to select one active Project Supervisor to receive the proposal, and shall notify that Supervisor.
- **REQ-PROPOSE-3** The system shall accept a proposal only during an Application Window.

4.13 ManageMatchingOutcome

4.13.1 Description and Priority

Description

Allows a Student to view published matching outcomes and accept or decline match offers.

Priority

High.

4.13.2 Stimulus/Response Sequences

1) Student navigates to "My Applications" and views the match offer details.
2) Student selects "Accept offer".
3) The System finalises the match assignment, registers the Student to the project, and withdraws the Student's other open applications.

Alternative Flow for Step 2

A. Manual Decline: Student selects "Decline". The System displays a confirmation warning regarding round disqualification, marks the offer as declined upon confirmation, and renders the Student ineligible for future rounds.

B. Auto-Decline: If the Student does not accept or decline the match offer before it expires, the System automatically transitions the match status to "Timed out".

4.13.3 Functional Requirements

- **REQ-OUTCOME-1** When a Student accepts a match offer, the system shall fill one slot of that listing, withdraw the Student's other open applications, and make the Student ineligible for later rounds.
- **REQ-OUTCOME-2** When a Student manually declines a match offer, the system shall render that Student ineligible to participate in any subsequent matching rounds for the current application cycle.
- **REQ-OUTCOME-3** If a Student does not respond before the offer expires (REQ-PUB-3), the system shall set the match to "Timed out".
- **REQ-OUTCOME-4** A Student whose match offer transitions to "Timed out" shall be rendered ineligible to participate in any subsequent matching rounds for the current application cycle.

4.14 ManageProfile

4.14.1 Description and Priority

Description

This feature allows a Student to complete, view, and update personal academic details, completed modules, research interests, and transcript documents.

Priority

High; required upon first sign-in and for maintaining accurate applicant data.

4.14.2 Stimulus/Response Sequences

1. The Student navigates to "My Profile".
2. The System displays the Student's school, CGPA, year of study, completed modules and research interests.
3. The Student updates fields and uploads an updated academic transcript PDF, then clicks "Save changes".
4. The System validates the file and inputs, then saves the profile.

4.14.3 Functional Requirements

- **REQ-PROF-1** The system shall require a Student to complete their profile after first sign-in.
- **REQ-PROF-2** The system shall allow a Student to enter their school, CGPA, year of study, completed modules and research interests.
- **REQ-PROF-3** The system shall accept supporting documents in PDF format up to 5 MB in size and reject invalid formats or sizes.

Project Supervisor Features (4.15 to 4.18)

4.15 ManageProjectDetails

4.15.1 Description and Priority

Description

This feature allows a Project Supervisor to create, update, and close research project listings.

Priority

High; required for maintaining active research offerings and managing project slots.

4.15.2 Stimulus/Response Sequences

1. The system displays the Project Supervisor's existing project listings and prompts them to select CREATE, UPDATE, or CLOSE.
   a) The Project Supervisor selects CREATE.
      i) The system prompts for the project title, description, prerequisites, number of available slots, and topic tags.
      ii) The Project Supervisor inputs the required details and selects PUBLISH or SAVE AS DRAFT.
   b) The Project Supervisor selects UPDATE.
      i) The system displays the selected project's existing details for editing.
      ii) The Project Supervisor modifies the details and selects SAVE.
   c) The Project Supervisor selects CLOSE.
      i) The system prompts the Project Supervisor to confirm closing the project.
      ii) On confirmation, the system marks the project listing as closed and removes it from student browsing.
2. The system validates the entered data and saves the listing as published or as a draft.

4.15.3 Functional Requirements

- **REQ-PROJ-1** The system shall allow a Project Supervisor to save a listing as a draft at any time, and to publish it before or during the application cycle but not after its closing date.
- **REQ-PROJ-2** The system shall only allow a project listing to be updated before the application closing date.

- **REQ-PROJ-3** The system shall prevent a Project Supervisor from reducing a project's slots below the number of applicants already matched to it.
- **REQ-PROJ-4** The system shall allow a Project Supervisor to close their own listing, removing it from student search.

4.16 ReviewAndRankStudentApplications

4.16.1 Description and Priority

Description

This feature allows a Project Supervisor to review students who applied to their project, filter applications, view top recommendations, and rank them in order of preference.

Priority

High; essential for evaluating applicant suitability and determining placement preferences.

4.16.2 Stimulus/Response Sequences

1. The Project Supervisor selects a project to review.
2. The system displays all applicants and a curated list of the top 10 recommended applicants based on supervisor filters (school, minimum CGPA and prerequisites).
3. The Project Supervisor selects an applicant to review their profile and transcript.
4. The Project Supervisor returns to the applicant list, arranges the applicants in order of preference, and submits their rankings.

4.16.3 Functional Requirements

- **REQ-VRANK-1** The system shall allow a Project Supervisor to filter applicants by school, minimum CGPA and whether prerequisites are met, and shall mark up to 10 recommended applicants: those meeting the prerequisites first, then by CGPA from highest.
- **REQ-VRANK-2** The system shall allow a Project Supervisor to submit and modify applicant rankings during the round's Application and Review Windows. Each applicant may appear once. Applicants left unranked shall not be matched to that listing.
- **REQ-VRANK-3** The system shall display applicant documents in the browser, only to the owner, Admins, and Supervisors of listings the Student applied to.

4.17 ApproveMatchingResults

4.17.1 Description and Priority

Description

This feature allows a Project Supervisor to review students proposed to be matched to their project and approve or reject each proposed match.

Priority

High; critical for finalising matches before administrative publication.

4.17.2 Stimulus/Response Sequences

1. The system displays the students proposed to be matched to the Project Supervisor's project.
2. The Project Supervisor selects a proposed match to review, and the system displays the student's profile and application details.
3. The Project Supervisor selects APPROVE, and the system records the match as approved and marks it as eligible for publication.
4. Alternatively, if the Project Supervisor selects REJECT, the system prompts for confirmation and a reason, records the rejection for Administrator review, returns the affected project slot to available, and notifies the Admins.

4.17.3 Functional Requirements

- **REQ-APPRV-1** The system shall allow a Project Supervisor to approve or reject only their own proposed matches, only after the matching process has been executed.
- **REQ-APPRV-2** When a proposed match is rejected, the system shall record the rejection for Admin review, return the affected project slot to available, and notify the Admins.
- **REQ-APPRV-3** The system shall prevent a Project Supervisor from approving or rejecting a proposed match after the matching results have been published.

4.18 ManageSelf-ProposedProjects

4.18.1 Description and Priority

Description

This feature allows a Project Supervisor to review research proposals pitched directly by students and either accept and register or decline them.

Priority

High; enables direct engagement with student-initiated research ideas.

4.18.2 Stimulus/Response Sequences

1. The Project Supervisor navigates to the "Proposals" page.
2. The System displays all proposals submitted by students directed to the supervisor.
3. The Project Supervisor reviews the proposal title, write-up, research tags, and supporting documents, then clicks "Accept and register".
4. The System registers the student under the supervisor and triggers a confirmation notification.
5. Alternatively, the Project Supervisor clicks "Decline", optionally adds a note, and the System notifies the Student.

4.18.3 Functional Requirements

- **REQ-REVPROP-1** The system shall allow a Project Supervisor to view self-proposed project submissions directed to them along with the proposing student's profile and supporting documents.

- **REQ-REVPROP-2** When a proposal is accepted, the system shall register the Student to a new one-slot project, withdraw the Student's other open applications, make the Student ineligible for later rounds, and notify the Student.
- **REQ-REVPROP-3** The system shall allow a Project Supervisor to accept a proposal only while the cycle is open, and to decline it at any time with an optional note.

User Features (4.19 to 4.21)

4.19 RegisterAccount

4.19.1 Description and Priority

Description

This feature allows a user to create a new account using an NTU email address and password, followed by an email verification process to activate the account.

Priority

High; essential for onboarding users into the system.

4.19.2 Stimulus/Response Sequences

1. The User navigates to the registration page.
2. The System displays input fields for full name, NTU email address, password, and a choice of Student or Project Supervisor.
3. The User enters details and clicks "Create account".
4. The System validates the email domain and checks for existing accounts, then sends a verification code to the provided email address.
5. The User enters the verification code, and the System verifies the code to activate the account.

4.19.3 Functional Requirements

- **REQ-REG-1** The system shall restrict registration to email addresses ending in an allowed domain, by default @e.ntu.edu.sg and @ntu.edu.sg. The list of allowed domains shall be configurable.
- **REQ-REG-2** The system shall require a user to enter the correct verification code to activate the account.
- **REQ-REG-3** The system shall reject a registration using an email address already associated with an existing account.
- **REQ-REG-4** The verification code shall be 6 digits and expire after 10 minutes, and the user shall be able to request a new one.
- **REQ-REG-5** Self-registration shall offer only the Student and Project Supervisor roles. Admin accounts are created by an Admin (REQ-USER-2).
- **REQ-REG-6** Passwords shall be at least 8 characters.

4.20 SignIn

4.20.1 Description and Priority

Description

This feature allows a registered, verified user to securely access the URECAMatch platform using their email address and password.

Priority

High; critical for authenticating users prior to system access.

4.20.2 Stimulus/Response Sequences

1. The User navigates to the sign-in page.
2. The System displays input fields for NTU email address and password.
3. The User enters their credentials and clicks "Sign in".
4. The System validates the credentials against the user database and redirects valid users to their respective dashboard.

4.20.3 Functional Requirements

- **REQ-AUTH-1** The system shall allow a verified user to sign in with their email address and password, with password input masked and an option to reveal it.
- **REQ-AUTH-2** The system shall prevent sign-in to a deactivated or unverified account.
- **REQ-AUTH-3** A user session shall expire after 30 minutes of inactivity.

4.21 ViewNotifications

4.21.1 Description and Priority

Description

This feature allows an authenticated user to view incoming and historical system notifications regarding status changes, application updates, cycle dates, and matching outcomes.

Priority

Medium; important for keeping users informed of critical updates.

4.21.2 Stimulus/Response Sequences

1. The User clicks on the "Notifications" tab in the navigation menu.
2. The System queries and displays the list of relevant timestamped notifications.
3. The User selects a notification to view its complete message details.
4. The System marks the selected notification as read and updates the unread badge counter.

4.21.3 Functional Requirements

- **REQ-NOTIF-1** The system shall deliver a notification only to the user it concerns and display notifications in reverse chronological order.
- **REQ-NOTIF-2** The system shall mark a notification as read when the user views it

5. Other Nonfunctional Requirements

5.1 Performance Requirements

5.1.1 Responsiveness

- During an Application Window, all URECA Project Listings (or a subset sufficient for loading a preliminary table display) shall be queried and fetched within 5 seconds.

- Basic visual feedback of interactive elements on the user interface shall be displayed within no more than 2 seconds after user interaction.

- A Supporting Document of up to 5 MB shall be accepted, validated and stored within 15 seconds of upload, with progress shown to the Student throughout.

5.1.2 Notification Latency

- In-app notifications shall appear within a minute of the relevant update, while email notifications shall be dispatched within a maximum of 15 minutes to the receiving users.

5.1.3 Capacity and Load

- The system shall accommodate up to 1000 concurrent active user sessions while continuing to meet the response times stated in 5.1.1.

- The system shall sustain the peak load expected in the final hour before an Application Window closes, without losing a submission or failing to serve the portal webpages to Users.

5.1.4 Matching Execution Time

- The matching process over the submitted Preference Rankings (from applying Students) and Applicant Rankings (from Project Supervisors) shall complete within 10 minutes from the conclusion of a Review Window.

- While a Matching Run is in progress, the system shall remain available for read-only access.

5.2 Safety Requirements

- All user preference and ranking data must be committed to persistent storage upon submission to ensure zero data loss in the event of a system crash.

- Following a system restart or unexpected crash, all core portal services and API integrations must recover and become accessible within 5 minutes.

- All Application and User data shall be backed up at least daily for the duration of an Application Cycle, and a backup shall be restorable to a working system within 4 hours.

5.3 Security Requirements

5.3.1 Authentication

- Registration shall be restricted to NTU email addresses (@e.ntu.edu.sg or @ntu.edu.sg), or approved external domains.

- A User session shall expire after 30 minutes of inactivity.

- Passwords shall be stored only as salted one-way hashes produced by a current password-hashing function, and shall never be written to logs, error messages or backups in recoverable form.

5.3.2 Authorization

- Every request shall be authorized against the requesting User's role, and any attempt to access a function outside that role shall be denied.

5.3.3 Personal Data Protection Act

- Personal data collected from Students, including CGPA, completed modules and Supporting Documents, shall be used only for the purpose of matching, and shall not be disclosed to any party other than the Admins and the Project Supervisors entitled to view it for the purposes of URECA applications.

- The handling of student records shall comply with the Singapore Personal Data Protection Act (PDPA) and with NTU's own policy on student data.

5.3.4 Audit Log Immutability

- Audit Log entries shall be append-only, with past records being strictly unmodifiable by a User with any role.

- Every Manual Reassignment shall be attributable to the Admin who performed it and to the time at which it occurred, and shall be retained for the lifetime of the Application Cycle's records.

5.4 Software Quality Attributes

5.4.1 Availability

- The portal shall maintain 99.9% monthly uptime during the official application and matching periods.

5.4.2 Usability

- The user interface must be fully responsive, rendering correctly on common screen sizes on desktop without content clipping.

- The system must provide visual feedback (e.g., loading spinners, progress bars, or "Success/Error" notifications) for all interactive elements, such as form submissions or ranking updates.

- All User-facing error messages must be written in plain, non-technical language and provide clear instructions on how to resolve the issue.

5.4.3 Maintainability, Reusability & Configurability

- The system shall allow for reusability across multiple application cycles, each taking place in its own respective academic year with simple configuration changes and minimal/no code modifications.

- Application Cycle dates, the length of the Pre-Application Window and the duration of each Matching Round's Application Window and Review Window shall be changeable by an Admin at run time, without redeployment.

5.4.4 Documentation

- The system shall be accompanied by maintained code documentation, API specification, and setup guide sufficient for a new developer to build and run it.

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

Business Rules

<List any operating principles about the product, such as which individuals or roles can perform which functions under specific circumstances. These are not functional requirements in themselves, but they may imply certain functional requirements to enforce the rules.>

5. Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

Appendix A: Glossary / Data Dictionary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Term	Description	Key Attributes	Relationships / Associations
URECA	Undergraduate Research Experience on Campus, a programme in which NTU undergraduate students undertake a research project supervised by a faculty staff or external mentor. Such projects can either be listed by the mentors, or self-proposed from the student.		URECAMatch administers URECA. URECA Projects are carried out under URECA.
URECAMatch (aka. the "System")	The system specified in this technical requirements document. Its primary goal is to provide a platform to facilitate an improved application process for Students, Project Supervisors, and Admins.		URECAMatch serves Admins, Project Supervisors and Students. URECAMatch hosts URECA Project Listings and Applications. One URECAMatch platform serves many User instances of all three types and hosts many URECA Project Listings. At any one time it runs at most one active Application Cycle.

URECA Project Listing	A URECAMatch descriptive entry of a URECA Project put forward by a Project Supervisor, and made available on the platform for Students to apply for. Comprises a project title, project description, (optional) prerequisites, topic tags, offering faculty/organization and main research location.	listingId, title, description, prerequisites, topicTags, availableSlots, filledSlots, offeringFaculty, researchLocation, coverImageUrl, status, createdAt	A Project Supervisor creates URECA Project Listings. A URECA Project Listing describes a URECA Project and receives Applications. One URECA Project Listing is created by exactly one Project Supervisor, while one Project Supervisor may create many listings. One URECA Project Listing describes exactly one URECA Project, and may receive many Applications or none. One URECA Project Listing may carry zero or more Topic Tags and zero or more Prerequisites.

URECA Project	A research project carried out by a Student under the mentorship of a Project Supervisor, typically over the span of 11 months in an NTU academic year. Typically, URECA Projects are put forwards by a Project Supervisor, but Students can also initiate a Self-Proposed URECA Project		A Project Supervisor supervises a URECA Project. Students are matched to URECA Projects. One URECA Project is supervised by exactly one Project Supervisor, while one Project Supervisor may supervise many URECA Projects. One URECA Project takes as many Students as its listing has slots, and one Student holds at most one URECA Project per Application Cycle.
Self-Proposed URECA Project (aka. “Self-Initiated URECA Project”)	A URECA Project proposed by a Student instead of being put forward by a Project Supervisor. Students can propose a URECA Project by submitting an Application for a Self-Proposed URECA Project, and subsequently contacting a Project Supervisor to approve the supervision of the project and/or provide additional resources if necessary.		A Self-Proposed URECA Project is a URECA Project. A Student proposes it to a Project Supervisor. One Self-Proposed URECA Project is proposed by exactly one Student and directed to exactly one Project Supervisor. One Student may submit many proposals during an Application Cycle but can be registered to at most one at the end of the Application Cycle.

User/Actor	A person operating an account on the URECAMatch platform. A User can be classified into 3 different types: Admin, Project Supervisor, and Student.	userId, email, name, role, accountStatus, createdAt, lastLoginAt	A User is an Admin, a Project Supervisor or a Student. A User owns a User Account and receives Notifications. One User holds exactly one role. Many Users of each role may exist in URECAMatch.
Verification Code	A 6-digit code sent to a user's NTU e-mail address to activate the account. It expires after 10 minutes, and a new one can be requested (REQ-REG-4).	code, expiresAt	A User is issued Verification Codes. Each Verification Code is issued to exactly one User, and one User may be issued zero or more.
Admin (aka. “Administrator”)	One of the 3 primary Users/Actors of the URECAMatch platform. Typically a member of the administrative staff overseeing the URECA Programme, the Admin shall have system-wide administrative privileges to manage Student and Project Supervisor accounts, configure application opening and closing dates, manually match projects to Students/Supervisors, notify Users, and modify various other records within the URECAMatch system.	Inherits the User Attributes, none of its own	Many Admins may exist. One Admin initiates zero or more Audit Log entries, and each entry is initiated by exactly one Admin.

Project Supervisor	One of the 3 primary Users/Actors of the URECAMatch platform. A Project Supervisor acts as the mentor in a URECA Project, and is able to supervise multiple URECA Projects and put forward these projects as URECA Project Listings on the platform. Project Supervisors are chiefly NTU faculty teaching staff, but can hail from external organizations as well.	faculty, organisation (plus the User attributes)	A Project Supervisor is a User. A Project Supervisor owns URECA Project Listings and supervises URECA Projects. One Project Supervisor may supervise many URECA Projects and Self-Proposed URECA Projects. One Project Supervisor may create multiple URECA Project Listings per Application Cycle. One Project Supervisor may own one Applicant Ranking for each URECA Project Listing he lists during each Matching Round. One Project Supervisor receives zero or more Self-Proposed URECA Applications.

Student	One of the 3 primary Users/Actors of the URECAMatch platform. A Student is a full-time NTU undergraduate who meets all eligibility requirements to be invited for the URECA application window, and can be accepted to at most one concurrent URECA Project.	school, cgpa, yearOfStudy, completedModules, researchInterests, isProfileComplete, isEligibleForMatching (plus the User attributes)	A Student is a User. A Student submits Applications during an Application Window and is matched to at most one URECA Project. One Student can be registered to at most one URECA Project per Application Cycle. One Student may submit multiple Applications to different URECA Project Listings during an Application Window, but only at most one Application to any given URECA Project Listing. One Student owns exactly one Preference Ranking per Matching Round.

Application	A Student's request to be considered for a URECA Project.	applicationId, statementOfInterest, submittedAt	An Application is either a Listing Application or a Self-Proposed URECA Application, and carries Supporting Documents. One Application is submitted by exactly one Student, and has exactly one Statement of Interest and zero or more Supporting Documents..
Match Result	The allocation of a Student to a URECA Project Listing for a given Matching Round. It is created as Proposed when matching runs, approved or rejected by the Project Supervisor, published by an Admin, and then accepted, declined or timed out by the Student (REQ-EXEC-5, REQ-REV-1).	resultId, status, preferenceRank, supervisorDecisionAt, publishedAt, offerExpiresAt, studentResponseAt, supervisorNote, justification	A Match Result pairs one Student with one URECA Project Listing, is published by an Admin after Project Supervisor approval, and is acted on by the Student. Each Match Result is associated with exactly one Student and one URECA Project Listing within exactly one Matching Round. A Student receives at most one Match Result per round, or none if their Application was unsuccessful. A Match Result may record the URECA Project Listing it was reassigned from.

Listing Application	An Application to a URECA Project Listing, as opposed to a Self-Proposed URECA Application.	preferenceRank, status (plus the Application attributes)	A Listing Application is an Application. One Listing Application is submitted to exactly one URECA Project Listing in exactly one Matching Round. One URECA Project Listing may receive many Listing Applications. Each Listing Application is ordered by at most one Applicant Ranking.
NTU	Nanyang Technological University, a tertiary higher education institution in Singapore. It hosts the URECA programme for its eligible undergraduates.		NTU hosts URECA. NTU enrols the Students and employs most Project Supervisors. NTU hosts exactly one URECA programme. It enrols many Students and employs many of the Project Supervisors, the remainder coming from external organisations.
CGPA	Cumulative Grade Point Average, (CGPA) the aggregate measure of a student's academic performance across all enrolled semesters on a scale from 0.00 to 5.00, with 5.00 being the highest	value	A CGPA is an attribute of a Student. A Project Supervisor views a Student's CGPA when ranking applicants. Each Student has exactly one CGPA.

	numerical grade point possible.		
Prerequisite	A condition stated on a URECA Project Listing describing the necessary background that a Student shall possess in order to be considered for a given URECA Project.		A URECA Project Listing states Prerequisites. A Prerequisite is assessed against a Student's completed modules. One URECA Project Listing states zero or more Prerequisites, and each Prerequisite belongs to exactly one listing.
Topic Tags	A short, key-word label attached to a listing (eg. “Artificial Intelligence” or “Finance”) that highlights the topics involved in the URECA Project. May be used for filtering search results.	Stored as the topicTags list of labels on URECA Project Listing; no separate class or ID	A URECA Project Listing carries Topic Tags. A Student filters listings by Topic Tag. One URECA Project Listing carries zero or more Topic Tags, and one Topic Tag may appear on many listings.
Statement of Interest	A brief free-text personal statement written by a Student when submitting an Application for a URECA Project Listing. Usually used to explain their motivation in applying for the project, as well as other relevant details not captured by the Supporting Documents.	body	An Application carries a Statement of Interest. A Student writes it and the Project Supervisor reads it. Each Application carries exactly one Statement of Interest, written by exactly one Student.

Supporting Document	A file uploaded by a Student in support of their Application, e.g. academic transcripts, resume, or CV.	documentId, fileName, fileFormat, fileSize, storagePath, uploadedAt	A Student uploads Supporting Documents. A Supporting Document may be attached to zero or more Applications. One Student may upload zero or more Supporting Documents, and each Supporting Document belongs to exactly one Student.
Application Cycle	A defined period set by Admins, in which Students can apply for projects listed by Project Supervisors, and Project Supervisors can consider and accept Students into their projects. During this period, Project Supervisors can also list new URECA Project Listings, and Students can similarly submit proposals for Self-Proposed URECA Projects. An Application Cycle is subdivided into a Pre-Application Window followed by one or more Matching Rounds; Students may submit Applications and proposals only during the Application Window of a Matching Round.	cycleId, academicYear, openingDate, closingDate, preApplicationDays, rolloverUnmatched, status	An Application Cycle opens with a Pre-Application Window and then runs up to three Matching Rounds. Applications belong to it through their Matching Round. One Application Cycle is created by exactly one Admin. It contains exactly one Pre-Application Window and up to three Matching Rounds.

Pre-Application Window	The opening period of an Application Cycle, during which Students may sign in and browse URECA Project Listings but may not submit Applications or Self-Proposed URECA Projects. Its length is configured by an Admin.	windowID, startDate, endDate, durationDays, status	An Application Cycle opens with a Pre-Application Window. A Pre-Application Window precedes the first Matching Round. A Student may view URECA Project Listings during it but may not apply. Each Application Cycle has exactly one Pre-Application Window, and each Pre-Application Window belongs to exactly one Application Cycle.
Matching Round	One complete pass of application, review and matching within an Application Cycle. A Matching Round runs an Application Window followed by a Review Window, after which the Matching Process is executed. An Application Cycle may run several Matching Rounds in sequence.	roundId, roundNumber, applicationWindowStart, applicationWindowEnd, reviewWindowStart, reviewWindowEnd, status, executedAt, publishedAt	An Application Cycle contains Matching Rounds. A Matching Round contains an Application Window and a Review Window, and ends with one execution of the Matching Process. A Student matched in an earlier Matching Round is excluded from later ones. One Application Cycle may contain up to 3 Matching Rounds, and each Matching Round belongs to exactly one Application Cycle. Each Matching Round has

			exactly one Application Window and exactly one Review Window. A Matching Round accepts Listing Applications, collects Applicant Rankings and produces Match Results.
Application Window	The first period of a Matching Round, during which Students may submit Applications and Self-Proposed URECA Projects and set their Preference Ranking. Its length in days is configured by an Admin.	startDate, endDate, durationDays	A Matching Round begins with an Application Window. A Student submits Applications and sets a Preference Ranking during it. An Admin configures its duration. Each Matching Round has exactly one Application Window, and each Application Window belongs to exactly one Matching Round.

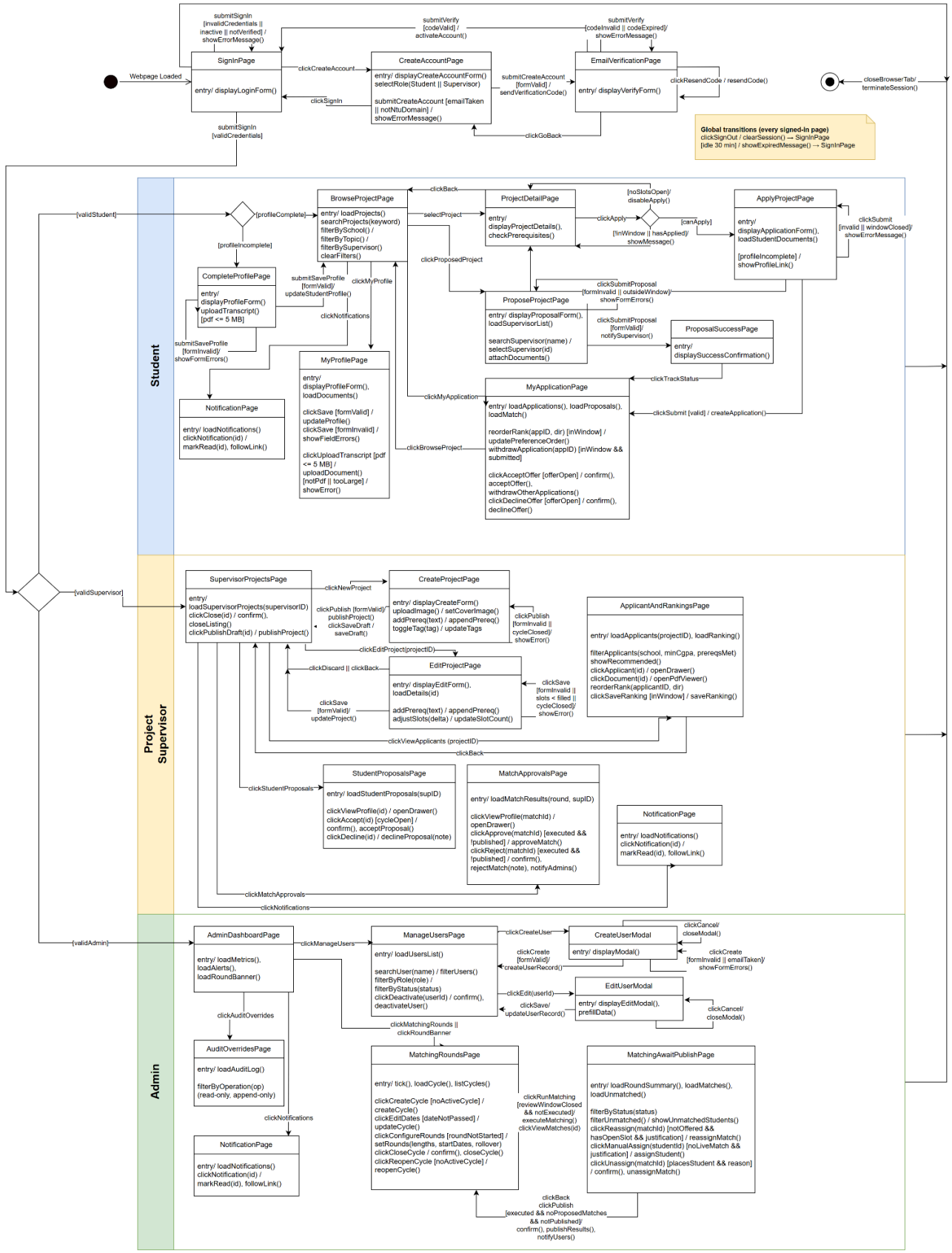
Review Window	The second period of a Matching Round, during which no new Applications may be submitted, and Project Supervisors review applicants, submit an Applicant Ranking, and decide on Self-Proposed URECA Projects. Its length in days is configured by an Admin.	startDate, endDate, durationDays	An Application Window is followed by a Review Window. A Project Supervisor submits an Applicant Ranking during it. An Admin configures its duration. Each Matching Round has exactly one Review Window. Many Applicant Rankings may be submitted during one Review Window, one for each URECA Project Listing that received Applications.
Preference Ranking	The ordering a Student gives to their own submitted Applications, from most to least preferred.	lastModifiedAt, rankedProjects	A Student owns a Preference Ranking, set during an Application Window. A Preference Ranking orders Applications and feeds the Matching Process. One Student owns exactly one Preference Ranking per Matching Round, and each Preference Ranking orders that Student's Applications in that round by choice preference.

Applicant Ranking	The ordering a Project Supervisor gives to the Students who applied to one of their listings. Serves as the Supervisor-side input to the Matching Process.	rankedApplicants, lastModified, submittedAt	A Project Supervisor produces an Applicant Ranking for a URECA Project Listing during the round's Application and Review Windows. An Applicant Ranking feeds the Matching Process. At most one Applicant Ranking is produced per URECA Project Listing in each Matching Round. It orders zero or more of the Listing Applications that listing received in that round, and applicants left unranked are not matched to it.
Self-Proposed URECA Application	A special type of Application, detailing a Student's request to be considered for a self-proposed URECA Project, instead of a normal Application to a URECA Project Listing.	title, description, researchInterests, status, decidedAt	A Student submits Self-Proposed URECA Application during an Application Window. One Self-Proposed URECA Application is submitted by submitted by exactly one Student to exactly one Project Supervisor, and has exactly one Statement of Interest and zero or more Supporting Documents, along with other details of

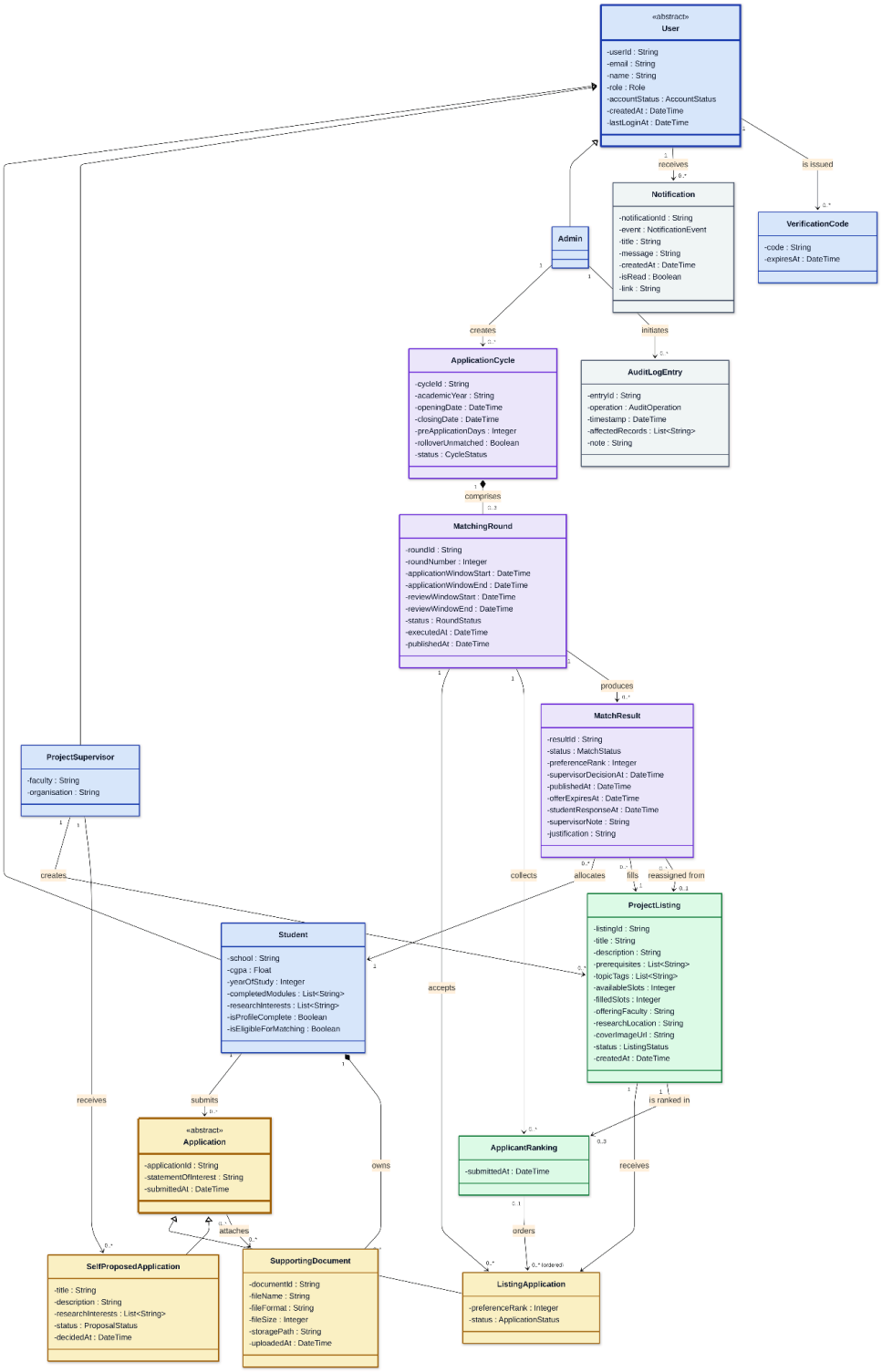
			the Self-Proposed URECA Project.
Audit Log	An append-only record of every administrative operation in URECAMatch	entryId, operation, timestamp, affectedRecords	Admins may view the Audit Log. No Users of any role are allowed to modify existing records on the Audit Log. Each administrative operation listed in REQ-AUDIT-1 creates one Audit Log entry, initiated by exactly one Admin.
Notification	A short summary of a System event sent to a specific User, meant to inform them of relevant changes on the URECAMatch portal.	notificationId, event, title, message, createdAt, isRead, link	A Notification is raised by a triggering event and delivered only to the User it concerns; a User views their Notifications in reverse chronological order, and each is marked read when its recipient views it. Each Notification is addressed to exactly one User; one User receives zero or more Notifications; each Notification arises from exactly one triggering event.

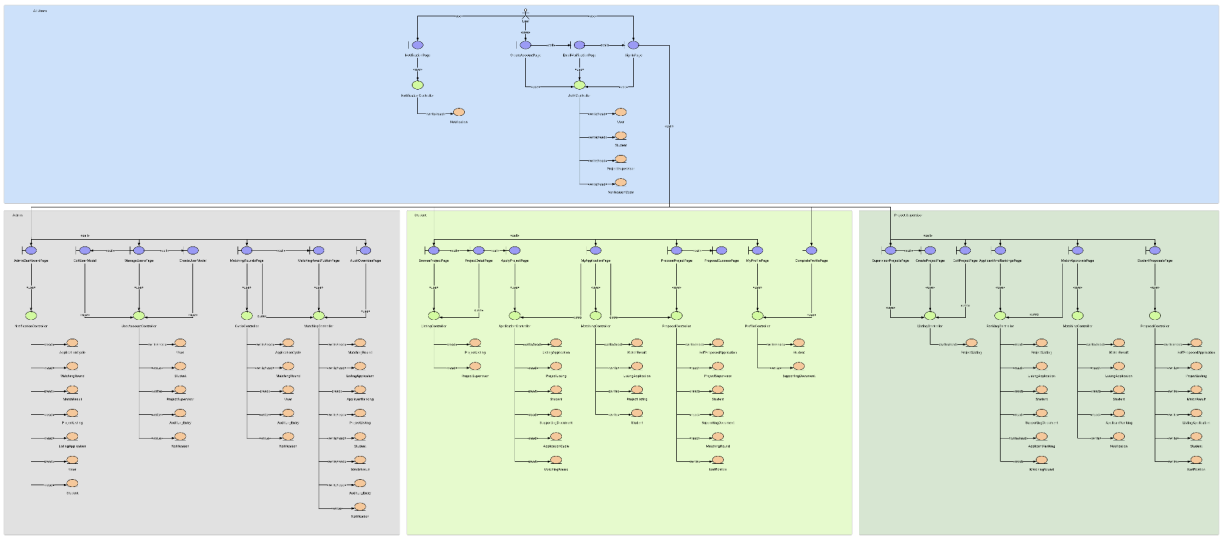
Appendix B: Dialog Map

URECAMatch Dialog Map



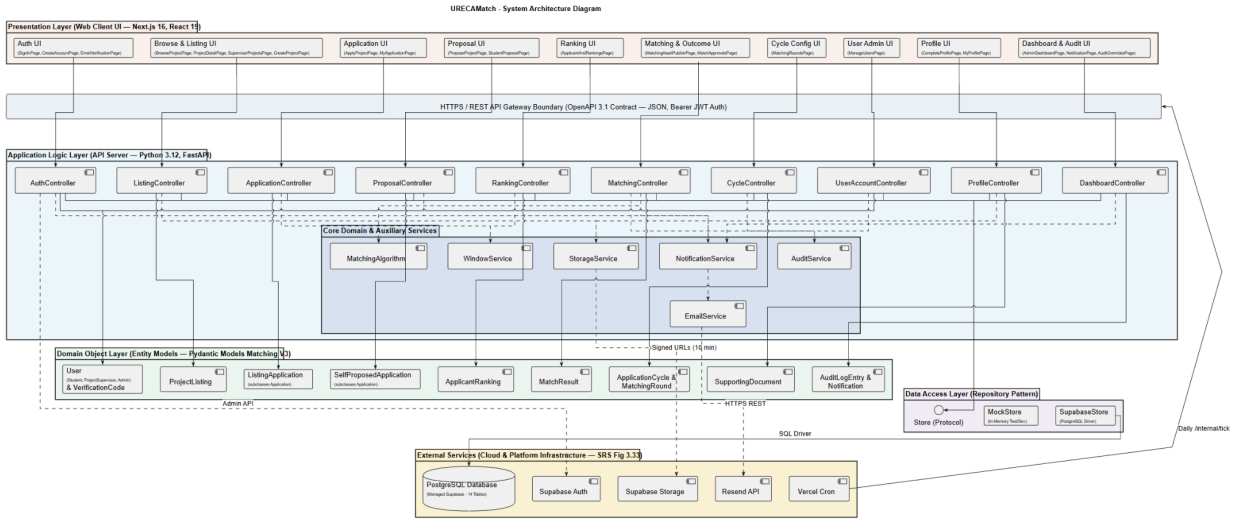
Appendix C: Class / Boundary Diagram





Appendix D: System Architecture Diagram

(simplified: main entity per controller)



Appendix E: Starting Skeleton Architecture (Refer: Lab 3 Supplementary AI Report)

```
URECAMatch/
├── README.md
├── .github/workflows/ci.yml   checks on every pull request
└── ureca-match-app/
    ├── api/
    │   ├── app/
    │   │   ├── main.py        entry point / driver
    │   │   ├── deps.py
    │   │   ├── errors.py      error functions/formatting
    │   │   ├── settings.py    configuration
    │   │   ├── routers/       10 routers (control)
    │   │   ├── services/      6 services (control)
    │   │   ├── models/        9 enums + 14 models (entity)
    │   │   └── db/            Store, seed data, export
    │   ├── api/index.py       Vercel entry point
    │   ├── tests/             28 test modules by requirement
    │   ├── pyproject.toml     Python dependencies
    │   ├── vercel.json        hosting and daily cron
    │   └── .env.example       environment settings
    ├── shared/openapi.json    generated API contract
    ├── supabase/              schema
    └── web/
        ├── app/               22 pages + 5 layouts (boundary)
        ├── components/        app shell, UI library, listing form
        ├── lib/
        │   ├── api/           ApiClient, live client, types
        │   ├── mock/          mock client
        │   ├── session.ts     session expiry
        │   └── use-api.ts     data hook for pages
        ├── e2e/               end-to-end test
        ├── package.json       web dependencies
        ├── next.config.ts     Next.js settings
        └── .env.example       environment settings
```

Appendix X: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>

Source:
http://www.frontiernet.net/~kwiegers/process_assets/srs_template.doc